



UNIVERSITY OF PADOVA

DEPARTMENT OF DEPARTMENT OF INFORMATION ENGINEERING

MASTER THESIS IN BIG DATA MANAGEMENT AND ANALYTICS

TRUSTWORTHY DATA-AWARE PROCESS SIMULATION FOR DECISION SUPPORT IN CONTAINER TERMINAL TRUCK OPERATIONS

SUPERVISOR

PROF. MASSIMILIANO DI LEONI
UNIVERSITY OF PADOVA

CO-SUPERVISORS

MARCEL PETERSEN
HHLA - PORT OF HAMBURG
PROF. ESTEBAN ZIMANYI
UNIVERSITÉ LIBRE DE BRUXELLES

MASTER CANDIDATE

JULE GRIGAT

STUDENT ID

2185651

ACADEMIC YEAR

2025-2026

TXAPELA BURUAN 'TA IBILI MUNDUAN: *RUE DE L'ÉTÉ, CARRER DE VIOLANT D'HONGRIA REINA
D'ARAGÓ, VIA LUIGI EINAUDI AND VIA FRANCESCO BELTRAME.*

Abstract

Contents

ABSTRACT	v
LIST OF FIGURES	xi
LIST OF TABLES	xiii
LISTING OF ACRONYMS	xv
1 INTRODUCTION	1
1.1 Motivation	1
1.2 Problem Statement	3
1.3 Research Objectives	3
1.3.1 Research Contributions	4
1.4 Research Questions	5
1.5 Thesis Outline	5
2 BACKGROUND AND RELATED WORK (20)	7
2.1 Container Terminal Operations	7
2.1.1 Container Terminals as Complex Logistics Systems	8
2.1.2 Truck Handling Processes	10
2.1.3 Simulation-Based Decision Support in Container Terminals	13
2.2 Operational Data and Event Logs	15
2.2.1 Transactional Information Systems	15
2.2.2 Events, Cases and Traces	15
2.2.3 Event Logs	18
2.3 Process Mining Foundations	19
2.3.1 Definitions	19
2.4 Business Process Simulation	24
2.4.1 Fundamentals of Business Process Simulation	24
2.4.2 Business Process Simulation Models	24
2.4.3 Simulation-Based Decision Support	26
2.4.4 Simulation Model Verification, Calibration and Validation	27
2.5 Simulation Discovery and Data-Aware Simulation	28
2.5.1 From Manual Simulation to Simulation Discovery	28
2.5.2 Discovery of Simulation Parameters	29

2.5.3	Data-Driven and Data-Aware Simulation [ONLY CASE LEVEL?]	30
2.6	Explainable and White-Box Simulation	32
2.6.1	Explainability in Simulation-Based Decision Support	33
2.6.2	Probabilistic White-Box Simulation Models	34
2.7	Simulation Quality, Validation/Assessment, and Trustworthiness	36
2.7.1	Dimensions of Simulation Quality	36
2.7.2	Simulation Validation	37
2.7.3	Trustworthiness of Automatically Discovered Simulation Models	40
2.7.4	Research Gap	40
3	CASE STUDY AND DATA ENGINEERING	43
3.1	Case Study Environment	44
3.1.1	HHLA Container Terminal Burchardkai	44
3.1.2	Data Sources	44
3.2	Event Log Engineering	45
3.2.1	Raw Data Sources and Preparation	45
3.2.2	Data Cleaning and Integration	48
3.2.3	Case Definition	50
3.2.4	Activity Construction	50
3.2.5	Activity Timestamps and Temporal Semantics	51
3.2.6	Context Enrichment	53
3.2.7	Final Event Log Structure	56
3.2.8	Event Log Characteristics	57
4	EXPERIMENTS AND EVALUATION FRAMEWORK	61
4.1	Research Questions, Hypotheses and Framing	63
4.1.1	Refined Research Questions	63
4.1.2	Hypotheses	63
4.2	Held-Out Train/Test Split	64
4.2.1	Rationale	64
4.2.2	Split Definition	64
4.2.3	Correction Relative to Earlier Drafts	65
4.3	Process Discovery Protocol	65
4.3.1	Algorithm	65
4.3.2	Why the Inductive-Miner Net Is Diagnostic Only	65
4.3.3	Conformance Protocol	66
4.4	Simulation Parameter Discovery Configurations	67
4.4.1	Configuration <i>no-rules</i>	67
4.4.2	Configuration <i>rules</i>	67
4.4.3	Configuration <i>rules+workload</i>	67
4.4.4	Discovery Hyperparameters	68

4.4.5	Authoritative Parameter Bundle and Serialization Contract	68
4.5	Monte-Carlo Replication Protocol	69
4.5.1	Replication Design	70
4.5.2	Implementation	70
4.6	Fidelity Metrics	70
4.6.1	Wasserstein Distance (EMD)	71
4.6.2	Kolmogorov–Smirnov Statistic	71
4.6.3	Point-Statistic Errors	72
4.6.4	Implementation	72
4.7	What-If Scenario Design	72
4.7.1	Scenario A: T22 Resource-Pool Reallocation	72
4.7.2	Scenario B: 20 % Higher Truck Demand	73
4.7.3	Final Domain-Constrained RMG Capacity	73
4.7.4	Reporting	74
4.8	Descriptive Transition-Duration Analysis	74
4.9	ProSiT Implementation Notes and Workarounds	75
4.9.1	Data-Attribute Discovery on NaN Columns	75
4.9.2	Simulation in No-Rules Mode	75
4.9.3	Duplicated Columns in the Simulated Log	75
4.9.4	Absence of Automatic Temporal Features	76
4.9.5	Non-Deterministic Simulation Engine	76
4.9.6	Contribution Boundary	76
5	RESULTS	77
5.1	Event Log Characteristics on the Held-Out Split	78
5.2	Process Discovery and Held-Out Conformance	78
5.3	Temporal Structure of the Arrival Process	79
5.4	Held-Out Simulation Fidelity (Multi-Seed)	80
5.4.1	RQ1: Point Estimate of Held-Out Fidelity	81
5.4.2	H2: Does the Data-Aware Configuration Improve Fidelity?	82
5.4.3	H3: Does the Workload-Feature Configuration Improve Fidelity?	82
5.4.4	Cross-Seed Variance Does Not Favour the Workload Variant	82
5.4.5	Methodological Note on Temporal Semantics	83
5.5	Per-Activity Decomposition of Fidelity	84
5.6	Freely Discovered Resource Representation and Final Domain Cap	85
5.7	Descriptive Analysis of Transition Durations	86
5.8	Final Calibrated Model and What-If Scenarios	87
5.8.1	Held-Out Baseline Interpretation	89
5.8.2	Scenario A: T22 Resource-Pool Reallocation	89
5.8.3	Scenario B: 20 % Higher Demand	90
5.9	Summary of Empirical Findings	91

5.10	Methodological Limitations	92
5.10.1	Model-Derived Waiting Time Includes Physical Movement	92
5.10.2	Resource Assignment Is Not Spatial Container Relocation	92
5.10.3	Systematic Turnaround Under-Prediction	93
5.10.4	Scenario Confidence Intervals Exclude Structural Uncertainty	93
5.11	Interpretive Conclusion of the Results Chapter	94
6	CONCLUSION (3)	95
6.1	Future Work	95
6.2	Personal Takeaways	95
	REFERENCES	97
	ACKNOWLEDGMENTS	101

Listing of figures

- 2.1 Two representations of the same discovered control flow (inductive miner on the CTB training log). The process tree exposes the hierarchical operator structure; the BPMN translation preserves the same behaviour using the industry-standard graphical vocabulary. Both models describe an identical set of allowed traces and are used interchangeably in the process mining literature [1]. 22

- 4.1 Experimental pipeline. *Data engineering* produces `s6_eventlog_target_rank_features.csv` (Chapter 3). The *split* of Section 4.2 separates the log into training and held-out testing partitions. *Process discovery* (Section 4.3) yields the Petri net used as the ProSiT skeleton. Three *simulation-parameter discovery* configurations (Section 4.4) are compared; each is evaluated against the held-out log through the *Monte-Carlo protocol* of Section 4.5. Two *what-if* experiments (Section 4.7) exercise the adopted-default configuration under operational disruption. . . . 62

- 5.1 CDF of the per-transition duration shift relative to the 5th percentile, over the 48 transitions with at least twenty observations. Illustrates the physical-movement component embedded in the model-derived pre-service delay. . . . 87

Listing of tables

2.1	Simplified Event Log Example	19
3.1	Activity labels used in the final event log.	51
3.2	Core process attributes.	56
3.3	Case-level container and process-shape attributes.	56
3.4	Demand and utilisation features at case arrival.	57
3.5	Target-area context attributes.	57
3.6	Temporal context attributes required by ProSiT.	58
3.7	Activity distribution in the final event log.	58
3.8	Final event log summary.	59
4.1	ProSiT discovery hyperparameters by configuration.	68
5.1	Activity counts on the train and test partitions.	78
5.2	Petri-net conformance on train and test.	79
5.3	Arrival-rate benchmark on 30-minute buckets (training log only, chronological 80/20 split).	80
5.4	Day vs. night arrival statistics on the training log.	80
5.5	Pre-calibration held-out screening by ProSiT configuration ($N_{\text{seeds}} = 10$; 95 % Student-t CI).	81
5.6	Cross-seed standard deviation of the fidelity metrics ($N_{\text{seeds}} = 10$).	83
5.7	Per-activity duration R^2 of a decision-tree model conditioned on target demand, target utilisation, target rank, visit complexity, receive/delivery counts, hour, weekday and weekend indicator (training log; grid search over depth/leaf, best per activity).	84
5.8	Resource representation in the freely discovered calibrated source bundle before applying the final RMG cap.	85
5.9	Empirical transition duration under alternative percentiles (seconds; 48 transitions with ≥ 20 observations).	86
5.10	Consolidated final validation and scenario KPIs ($N_{\text{seeds}} = 10$, seeds 42–51; 17,892 cases per run). Pre-service denotes the model-derived interval from enablement to activity start, not pure queueing time. “Observed” arrival measures use the minute-resolved simulated event log and elapsed calendar time.	88
5.11	Held-out service-time EMD by yard activity for the final cap-three baseline ($N_{\text{seeds}} = 10$). Gate In and Gate Out are excluded.	89

5.12	Summary of empirical findings.	91
------	--	----

Listing of acronyms

BPMN	Business Process Model and Notation
CHE	Container Handling Equipment
CRM	Customer Relationship Management
CTB	HHLA Container Terminal Burchardkai
DES	Discrete Event Simulation
DFG	Directly-Follows Graph
ERP	Enterprise Resource Planning
GOS	Gate Operating System
HHLA	Hamburger Hafen und Logistik AG
HO ₂	Manual yard handling area
KPI	Key Performance Indicator
LL	Leerlager (empty-container yard)
MES	Manufacturing Execution System
OCR	Optical Character Recognition
PM	Process Mining
RMG	Rail-Mounted Gantry Crane
TAS	Truck Appointment System
TCS	Terminal Control System
TEU	Twenty-foot Equivalent Unit
TOS	Terminal Operating System
TTT	Truck Turnaround Time

VC Van Carrier
WfMS Workflow Management System
XES eXtensible Event Stream
YMS Yard Management System

1

Introduction

Container terminals are critical nodes within global supply chains and play a key role in ensuring the efficient movement of goods between maritime and hinterland transportation networks. Increasing container volumes, larger vessel sizes, and growing operational complexity require terminal operators to make numerous tactical and operational decisions under uncertain conditions. Examples include truck dispatching, yard block allocation, resource utilization, congestion management, and the handling of disruptions such as equipment failures or temporary infrastructure outages. Traditional experimentation in real terminal environments is often costly, disruptive, or operationally infeasible. Therefore, simulation has become a widely used decision-support technique in terminal planning and operations research.

1.1 MOTIVATION

Simulation is a widely used decision-support technique in container terminal planning and operations research. Due to the operational continuity of container terminals, experimentation in real environments is generally costly, disruptive, and associated with considerable operational risk. Consequently, simulation models are frequently employed to evaluate infrastructure investments, operational policies, scheduling approaches, truck appointment systems, and terminal redesign alternatives before implementation. Traditional approaches predominantly rely

on manually developed Discrete Event Simulation (DES) models, agent-based models, or digital twins that require substantial modeling effort, expert knowledge, and continuous calibration to accurately represent real-world terminal operations [2, 3, 4, 5].

In parallel, the Process Mining community has investigated methods for automatically extracting process knowledge from event logs generated by operational information systems. One long-standing research objective has been the automated construction of simulation models directly from process execution data. Early work demonstrated that simulation-relevant information, including control-flow behavior, activity durations, arrival patterns, resource allocations, and routing probabilities, can be automatically discovered and incorporated into executable simulation models [6]. Recent advances have extended this idea toward *data-aware process simulation*. Instead of relying solely on control-flow information and static branching probabilities, these approaches incorporate contextual process attributes, case-specific information, process states, resource characteristics, and decision logic into automatically discovered simulation models [7, 8, 9, 10].

Research has increasingly pointed to the limitations of black-box simulation approaches. While machine learning and deep learning methods are able to learn complex behavioral patterns, they often provide limited transparency and are difficult to inspect, modify, and validate when used for operational decision support. Consequently, recent studies advocate hybrid and white-box approaches that preserve explainability while exploiting data-driven behavioral discovery [11, 12]. Frameworks such as ProSiT advocate explainable and configurable simulation models based on probabilistic white-box predictors, with the explicit goal of supporting what-if analyses while maintaining model transparency and interpretability [12].

Despite these advances, empirical evidence regarding the applicability of automatically discovered simulation models in logistics systems remains limited. Existing evaluations primarily focus on business-process environments such as healthcare, financial services, and administrative processes [13, 9]. Container terminals and truck operations, on the other hand, are characterized by physical resource constraints, congestion effects, infrastructure dependencies, spatial interactions, and highly dynamic process contexts. Whether automatically discovered data-aware simulation models can adequately represent these characteristics and provide trustworthy decision support remains largely unexplored. This thesis addresses this research gap by investigating the applicability, realism, robustness, and decision-support capability of data-aware process simulation models discovered from real container terminal truck operation event logs.

1.2 PROBLEM STATEMENT

Container terminal operators continuously face decisions regarding capacity management, truck handling processes, yard utilization, operational policies, and infrastructure changes. Simulation models are often used to evaluate such decisions because experimentation in the real terminal environment is often infeasible or associated with significant operational risks. However, most existing container terminal simulation studies rely on manually developed simulation models. These models require substantial domain expertise and modeling effort and are often costly to maintain as terminal operations evolve. Furthermore, manually constructed models may introduce subjective assumptions regarding process behavior, resource interactions, and routing logic [3, 4]. Data-aware process simulation offers a fundamentally different paradigm. Instead of manually designing simulation behavior, the simulation model is automatically discovered from historical event logs. This approach promises improved scalability, reduced modeling effort, and closer alignment with observed process behavior [14, 10, 9]. However, the practicality of this approach implicitly depends on several assumptions. It must be demonstrated that automatically discovered models can adequately reproduce observed operational behavior, that they remain interpretable and configurable, and that they provide sufficiently robust simulation outcomes to support decision-making. While previous studies have demonstrated the technical feasibility of data-aware simulation in business-process environments [9, 13], it remains unclear whether comparable results can be achieved in logistics systems characterized by congestion effects, resource competition, and highly context-dependent behavior. Therefore, a systematic evaluation of the trustworthiness and limitations of automatically discovered data-aware simulation models in container terminal operations and regarding this thesis in the subsystem of truck operations is required.

1.3 RESEARCH OBJECTIVES

The primary objective of this thesis is to evaluate whether automatically discovered data-aware process simulation models can provide trustworthy and robust decision support in the context of container terminal truck operations. More specifically, the thesis aims to:

- assess the ability of automatically discovered simulation models to reproduce observed operational behavior;

- investigate the modeling challenges that emerge when applying data-aware simulation discovery techniques to container terminal processes;
- analyze how alternative modeling assumptions affect simulation outcomes and resulting decision recommendations;
- evaluate the suitability of discovered simulation models for operational what-if analyses and previously unobserved scenarios.

The study focuses on truck handling operations within a container terminal environment and uses real operational event data as the basis for process discovery and simulation model generation.

1.3.1 RESEARCH CONTRIBUTIONS

The expected scientific contributions of this thesis are as followed:

1. Domain Evaluation Contribution
The thesis provides one of the first evaluations of data-aware process simulation in the domain of container terminal truck operations.
2. Methodological Contribution
The thesis identifies domain-specific challenges and limitations that emerge when transferring data-aware simulation approaches from traditional business-process environments to logistics systems with physical constraints and congestion effects.
3. Robustness and Trustworthiness Contribution
The thesis investigates how alternative plausible specifications of resources, arrival processes, and contextual attributes influence simulation outcomes and decision-support recommendations.
4. Decision-Support Contribution
The thesis evaluates the suitability of automatically discovered simulation models for operational what-if analyses involving scenarios not present in the original event log.

1.4 RESEARCH QUESTIONS

To achieve these objectives, the thesis addresses the following research questions:

1. To what extent can a data-aware process simulation model discovered from historical container terminal event logs reproduce observed operational behavior?
2. Which modeling challenges and limitations emerge when applying data-aware simulation discovery techniques to container terminal truck handling operations?
3. To what extent do alternative plausible specifications of discovered resources, arrival processes, and contextual process attributes affect simulation outcomes and decision recommendations?
4. To what extent can automatically discovered simulation models provide reliable decision-support recommendations for operational scenarios not represented in the historical event log?

1.5 THESIS OUTLINE

The remainder of this thesis is organised into six chapters. Chapter 2 reviews the theoretical foundation: container-terminal operations, process mining, business-process simulation and the state of the art in data-aware and white-box simulation methods, together with existing approaches for simulation validation and trustworthiness assessment.

Chapter 3 documents the case study environment (HHLA Container Terminal Burchardkai), the raw operational data sources and the deterministic event-log-engineering pipeline that turns the raw records into the simulation-ready log used throughout the remainder of the thesis.

Chapter 4 defines the experimental framework. It refines the research questions into falsifiable hypotheses, specifies the held-out temporal train/test split, describes the process-discovery and ProSiT-based simulation-discovery protocols, defines the Monte-Carlo replication scheme used to produce confidence intervals, defines the fidelity metrics, describes the two operational what-if scenarios, a descriptive transition-duration analysis, and documents the ProSiT implementation workarounds required for the CTB event log.

Chapter 5 reports the empirical evidence: descriptive characteristics of the train/test partitions, Petri-net conformance on both partitions, the temporal structure of the arrival process, the held-out fidelity of the three ProSiT configurations with multi-seed confidence intervals, the per-activity decomposition of fidelity, the discovered resource capacities, a descriptive analysis of transition durations, and the two what-if scenarios. Every empirical claim is qualified by its confidence interval or its inherent uncertainty.

Chapter 6 concludes the thesis by consolidating the findings against the research questions, discussing methodological limitations and outlining directions for future work.

2

Background and Related Work (20)

2.1 CONTAINER TERMINAL OPERATIONS

Container terminals represent critical nodes within global supply chains and facilitate the transition between maritime and Hinterland transportation networks. [seaport erläutderung!] They serve as interfaces between deep-sea vessels and hinterland transportation systems including road, rail, and inland waterway transport. Modern container vessels have a capacity of more than 20,000 containers in a single voyage, requiring container terminals to process large cargo volumes efficiently while maintaining short turnaround times and high service quality [SOURCE]. Since their introduction in the 1960s, standardized containers have become the dominant unit for global freight transportation. To ensure comparability across different container sizes, container throughput is commonly measured in Twenty-foot Equivalent Units (TEU). A standard 20-foot container corresponds to one TEU, whereas a 40-foot container corresponds to two TEUs [15]. Container terminals operate under considerable operational pressure. Increasing international trade volumes, larger vessels, and growing demands for rapid cargo handling require terminal operators to coordinate numerous interdependent processes and resources simultaneously. As a result, operational decisions directly affect vessel berth times, truck turnaround times, a terminals throughput, and overall service quality [3, 4]. The empirical context of this thesis is the HHLA Container Terminal Burchardkai (CTB) located in the Port of Ham-

burg. The CTB is the largest container terminal in Hamburg and. The terminal handles large volumes of import, export, and transshipment containers while coordinating interactions between vessels, trucks, rail services, storage yards, and container handling equipment. [ACTUAL NUMBERS] 6.3 mio HHLA total [Abbildung Hinterland/ Wasserseite Abgrenzung Bild von der HHLA]

2.1.1 CONTAINER TERMINALS AS COMPLEX LOGISTICS SYSTEMS

Container terminals are handled as highly complex logistics systems due to the large number of interacting resources, transportation modes, and operational dependencies involved in the everyday operations. [3, 4].

CONTAINER FLOWS

Within a seaport container terminal, containers move between multiple transportation modes and storage locations. A container may arrive by deep-sea vessel, feeder vessel, truck, or rail and subsequently leave the terminal using any of these transport modes. [HINTERLAND ERWÄHNEN] Consequently, a terminal continuously coordinates container flows between the seaside and the landside of the supply chain [15]. Hartmann distinguishes four primary transportation modes within container terminal operations: deep-sea vessels, feeder vessels, trucks, and rail services. Together, these modes create numerous possible transport paths through the terminal and generate highly dynamic container flows that must be coordinated efficiently [15]. Most containers entering a terminal undergo at least one temporary storage phase within the container yard before being transferred to their outbound transportation mode. Therefore, terminal operations must coordinate not only transportation processes but also inventory management and storage allocation decisions [3]. Figure: Container Flow through a Container Terminal [Wasserseite -> LAGER -> hINTERLAND Günstig in Bezug auf Hamburger Ladungsverteilung]

YARD COMPONENTS AND RESOURCES

Container terminals consist of multiple operational subsystems that interact continuously. Bett et al. distinguish three primary terminal areas: the quayside, the yard area, and the landside. The quayside includes berths and quay cranes used for vessel loading and unloading. The yard acts as an intermediate storage area for containers, while the landside connects the terminal to

trucks and rail operations[5]. For this Thesis relevant is the terminals yard area, that splits into three separate areas in the case of the CTB in Hamburg:

1. Automatic storage Blocks: The majority of containers is stored in the automatic storage blocks, operated by "Rail Mounted Gantry Cranes" - therefore often called RMG-Blocks. The CTB operates 22 RMG-Blocks with a total capacity of 2231 operative TEU slots and 3 RMGs per Block, operating between the waterside and the landside.[total of 40.000 operative TEU in Block] up to 5 high due to operational moves.
2. Manual Yard: In addition to the automated storage area, containers can be stored in the manually operated yard. Due to its central location within the terminal, this area is frequently used for operationally flexible storage depending on the assigned vessel berth, yard workload, and equipment availability. Container movements are typically performed by manually operated handling equipment.
3. Empty Container Yard: Empty containers are primarily stored in a dedicated empty container yard. Unlike full containers, empty containers are generally not assigned to a specific customer order but are managed as interchangeable inventory within a shipping line's container fleet. We want containers whose auslieferung noz numerical is, we want to store them as dense as possible, different handling equipment, and up to 6 high. Storing empty containers separately prevents them from occupying or being stacked on top of storage positions of full containers that are linked to specific transport orders. The empty container yard is operated using reach stackers, allowing storage heights of up to six containers.

hhla WEBSEITE BILD VOM BURCHARDKAI

Trucks can be ordered to pick up and deliver their cargo at all of these yard sections, waiting in transfer points to be operated by the respective resources. Resource shortages or inefficiencies within one subsystem often show effects throughout the whole terminal and negatively impact other operations [3].

INTERDEPENDENCIES BETWEEN TERMINAL PROCESSES

An important characteristic of container terminals is the strong interdependency of their operational processes. Individual decisions rarely affect only one subsystem. Instead, local disturbances often propagate throughout the entire terminal. [For example, delayed vessel arrivals

may postpone container unloading operations, which subsequently affects yard occupancy levels. Increased yard congestion can reduce crane productivity and generate longer waiting times for trucks arriving at the terminal. These delays may ultimately result in congestion at terminal gates and reduced service quality for trucking companies] - Mit Fokus, was bedeutet das eigentlich tatsächlich für ein terminal? [3, 5]. The complexity of these interactions is one of the primary reasons why analytical optimization approaches alone are often insufficient for evaluating operational decisions in container terminals.

2.1.2 TRUCK HANDLING PROCESSES

IMPORTANCE OF TRUCK OPERATIONS

Road transportation is a major component of hinterland connectivity in modern container terminals. As a consequence, terminals must coordinate large numbers of truck arrivals while maintaining acceptable service levels and avoiding excessive congestion and effects on public traffic in the port area [3, 16]. Fluctuating truck arrival patterns can exceed the available processing capacity of gates, yard equipment, and storage areas, resulting in longer waiting times and reduced terminal efficiency [5, 16].

A [TYPICAL] TRUCK VISIT PROCESS

To regulate truck arrivals and reduce congestion, many container terminals employ a Truck Appointment System (TAS). [Thats important as a decision support resulting tool!] Within a TAS, trucking companies reserve a predefined time slot before visiting the terminal and specify the container transactions that will be executed during the visit [5, 16]. A typical truck visit begins with the reservation of an appointment slot and the submission of the corresponding transport order. When arriving at the terminal, the truck passes through an automated pre-gate area where Optical Character Recognition (OCR) systems identify containers, vehicle registration plates, and aggregates touring information with booking information. [Despite the hourly booking slots, the Depending on the current workload and gate queue lengths, the truck may have Referr to Poisson later in the arrival distribution section]to wait before being processed at the physical gate and officially entering the terminal area [5]. After entering the terminal, the truck drives to the assigned container yard location. Depending on the transport order, the truck can perform one or multiple container transactions. These transactions may include:

- delivering an export container,

- receiving an import container,
- delivering or receiving multiple containers while at the same yard location,

A particularly efficient operational pattern is referred to as a *multi move*. In a multi move, a truck delivers one or more export containers and subsequently picks up one or more import containers during the same terminal visit. Multi moves reduce empty truck trips, improve truck utilization, lower emissions, and decrease congestion both inside and outside the terminal, but require efficient routing by the terminal [16]. Following the completion of all assigned transactions, the truck proceeds to the exit gate, completes the required departure procedures, and leaves the terminal. The total time spent between entering and leaving the terminal is commonly referred to as the *Truck Turnaround Time (TTT)*, which represents one of the most important performance indicators in truck and overall terminal operations [5].

TERMINAL OPERATING SYSTEMS

Modern container terminals rely on integrated information systems to coordinate operational activities. At the center of these systems are the Order Management and Terminal Control System (TCS), that orchestrate the movements and storage of containers across terminal subsystems and manages operational orders throughout the terminal lifecycle. The TOS is typically connected to specialized subsystems including a Gate Operating Systems (GOS), Yard Management Systems, Equipment Control Systems, and appointment management platforms. Together, these systems continuously generate and manage operational event data whenever trucks arrive, containers are handled, orders are updated, or terminal resources execute work instructions. Each subsystem stores a different subset of operational information. While gate systems primarily record arrival and departure events and details, yard management systems capture container movements, assign storage locations and and communicates with the equipment control system thats responsible for the equipment execution orders. Consequently, the complete reconstruction of truck visits often requires the integration of multiple operational data sources. For the purposes of this thesis, the Order and Terminal Control System serves as the primary data source. Additional information from other operational systems is subsequently used to enrich and validate the resulting event log. The employed data integration process is described in detail in Chapter 4.

SOURCES OF OPERATIONAL DELAYS

Truck turnaround times are influenced by various operational factors that function individually or interdependent throughout the terminal processes. Common reasons for delay include:

- congestion at entry and exit gates, also caused and affected by road traffic outside the terminal
- pick up containers that are buried due to unexpected long dwell time on the terminal
- truck traffic congestion within container yard sections and main routes,
- limited availability of container handling equipment (CHE),
- high occupancy levels within storage blocks,
- resource and priority conflicts between truck and vessel operations,
- unexpected equipment failures or maintenance activities

Because these factors are highly interdependent, local disruptions often affect the entire terminal operations. For example, insufficient yard crane availability and a prioritised vessel departure in time can increase truck service time, which affects the trucks waiting times in the yard, which subsequently results in traffic congestion on the terminal area, a lower gate throughput and overall slowed Hinterland performance [3, 5].

OPERATIONAL PERFORMANCE INDICATORS

The performance of truck operations is commonly evaluated using a set of operational key performance indicators (KPIs).

- Truck Turnaround Time (TTT): Total time a truck spends inside the terminal between entry and exit.
- Waiting Time: Time spent waiting for gate processing, yard equipment, or storage area access.
- Yard Utilization: Degree to which storage areas are occupied and operational resources are utilized.

Among these indicators, Truck Turnaround Time is frequently considered the most important measure of truck service quality because it directly reflects the operational efficiency experienced by trucking companies and cargo owners and is therefore one out of four main KPIS for the HHLA Container Terminals.[5, 16].

2.1.3 SIMULATION-BASED DECISION SUPPORT IN CONTAINER TERMINALS

This is now trying to explain Why is there a need for better decision support in container terminals in general, why is simulation a common approach for that purpose, and why are traditional simulation approaches insufficient?

The Management of Container Terminals continuously faces operational decision making challenges. Decisions regarding truck appointment policies, resource allocation, equipment scheduling, maintenance planning, and yard management need to be made while operations remain active. Because container terminal processes are very interconnected, local decisions can quickly affect other parts of the system and influence waiting times, resource utilization, throughput, and eventually customer service quality. Although terminal managers often possess extensive operational experience, many decisions involve considerable uncertainty. Changes to operational procedures or capacity allocations may have unintended consequences, create congestion, increase handling times, or reduce service reliability. As a result, decision-making in container terminals increasingly relies on analytical and digital decision-support approaches that allow operational alternatives to be assessed before implementation. Simulation as a decision support tool in container terminal management has become an interesting and widely adopted research field. The application in reality is still lacking though, due to high complexity, costs, and a missing technical expertise or priority in the terminal teams. Running a Simulation enables managers and researchers to evaluate operational scenarios in a virtual environment without affecting live terminal operations. By reproducing the interactions between trucks, handling equipment, storage areas, and gate processes, simulation can estimate the effects of operational changes before they are implemented in practice. Typical applications of simulation in container terminals include the evaluation of Truck Appointment Systems (TAS), equipment allocation strategies, resource planning, yard management and storage placement strategies. Simulation is especially valuable when analysing congestion effects, resource bottlenecks, and operational trade-offs that are difficult to evaluate based on purely critical analytical thinking. A variety of studies have demonstrated the usefulness of simulation for improving truck turnaround times, resource utilization, and overall terminal performance [17, 5, 18].

Various simulation paradigms have been applied in terminal research. Discrete Event Simulation (DES) is the most frequently used approach because container terminal operations can naturally be represented as sequences of discrete operational events such as truck arrivals, gate processing, container handling, and equipment usage. Other approaches such as System Dynamics, Agent-Based Modeling, and microscopic traffic simulation have also been applied to investigate specific operational and strategic questions [19, 20, 18]. A more detailed discussion of simulation methodologies is provided in Section 2.3. Despite their widespread adoption, traditional simulation projects suffer from several limitations. Developing a simulation model typically requires extensive domain knowledge, detailed process understanding, and considerable modeling effort. The collection, formalization, implementation, calibration, and validation of process knowledge often represent the most time-consuming phases of a simulation project. Furthermore, manually developed simulation models most of the time rely on expert assumptions regarding process behavior. However, experts often describe how processes are intended to operate rather than how they are actually executed in practice, therefore, unintended discrepancies can appear between the simulation model and the real operational processes [6].

In dynamic environments like container terminals, these differences can impact the quality of simulation results. Another challenge is the continuous maintenance and calibration effort required to keep simulation models aligned with changing operational conditions. As terminal processes, workloads, and operating rules evolve over time, manually maintaining simulation models becomes costly and requires technical support. At the same time, modern container terminals continuously record large amounts of operational event data through Terminal Operating Systems, Gate Operating Systems, and related information systems. These data sources provide detailed information about actual process executions and operational behavior. Rather than relying mainly on expert knowledge, process knowledge could be extracted directly from operational data. Process Mining provides a methodological framework for discovering, analysing, and monitoring real processes based on historic operational data. By deriving process information directly from recorded datasets, Process Mining offers the potential to reduce modeling efforts, while improving the representation of real-world behaviour. Consequently, it represents a promising foundation for data-driven simulation model construction and automated simulation model discovery.

2.2 OPERATIONAL DATA AND EVENT LOGS

2.2.1 TRANSACTIONAL INFORMATION SYSTEMS

Woraus und warum entstehen überall überhaupt Eventlogs? Today, most business processes are supported and documented through information systems. These systems are primarily designed to facilitate and document operational activities rather than to analyze whole process flows. Examples include Enterprise Resource Planning (ERP) systems, Customer Relationship Management (CRM) systems, Manufacturing Execution Systems (MES), and Terminal Operating Systems (TOS) used in container terminals. Whenever users or objects interact with such systems, digital records are generated and stored. Activities such as creating, updating, processing, or completing an order leave traces in the underlying databases. While this information is mainly required for operational control, documentation, and basic analysis purposes, it simultaneously creates detailed records of (partial) process executions. In container terminals, operational data are continuously generated by systems such as the Terminal Operating System (TOS), Gate Operating System (GOS), and Yard Management System (YMS). Typical examples include the registration of a truck arrival at a gate, the assignment of a container to a storage location, or the execution of a container handling operation. Each transaction recorded by these systems creates information about what happened, when it happened, and which operational object was involved. As those kind of business processes are executed step by step, these recorded transactions form chronological sequences of events. Such event data provide the foundation for Process Mining techniques, which aim to reconstruct and analyze actual process behavior directly from system-generated data. Regarding the HHLA-CTB information systems it should be noted, that due to historical growth and development of the individual systems, not all systems collect the same level of detail or have alternating names for the same facilities, which will be elaborated on when discussing the event log engineering.

2.2.2 EVENTS, CASES AND TRACES

Explanation of Event Case Trace Activity Timestamp Attribute Using HHLA Examples

Process Mining views business processes as sequences of events recorded during one process execution. To describe these sequences, several fundamental concepts should be explained.

EVENT An event represents the occurrence of a specific activity within a process at a particular point in time. Each event documents that something happened during the execution of a process. Examples of events in a container terminal include:

- Truck drives through OCR gate,
- Gate Check completed,
- Truck arrives at yard position,
- Container operation at yard position completed,
- Truck exits the terminal.

ACTIVITY An activity describes the type of action that is being executed. Multiple events can belong to the same activity while occurring at different times or for different process instances. Examples of activities are:

- Gate In,
- Container-Delivery,
- Container-Receive,
- Gate Out.

CASE A case represents a single process instance. It is defined by the object whose process execution is being analyzed. Depending on the application domain, a case can represent different entities, such as:

- a customer order,
- a container,
- a truck visit.

In the context of this thesis, a truck visit serves as the primary process instance and therefore represents the case that is being investigated in.

In traditional event logs, events are often represented by a single timestamp, typically indicating the completion of an activity. For performance-oriented Process Mining and simulation

applications, richer temporal information is required. Two directly observable lifecycle states are commonly distinguished:

- Start Timestamp: The moment execution of the activity actually begins.
- Complete Timestamp: The moment execution of the activity finishes.

The duration between these two timestamps is the *service time* (or execution time) of the activity. A third temporal concept, *enablement*, describes the moment at which an activity becomes executable according to the process-model state. In process-mining-based simulation approaches, enablement is typically not an observed input-log attribute but is derived by replaying the event log against the discovered process model [6, 14, 12]. The interval between model-derived enablement and the observed start timestamp is referred to as *waiting time* in the ProSiT framework [12] and corresponds to a pre-service delay that may include resource contention, queueing effects, and — in domains with spatial separation between activities — physical movement time.

ATTRIBUTES In addition to the mandatory process information, events may contain supplementary attributes that provide further context about the execution of the process. Examples of event attributes in container terminal operations include:

- Truck ID,
- Container Number,
- Shipping Line,
- Yard Block,
- Container Type,
- Equipment Identifier,
- Trucking Company.

These attributes enable more detailed analyses and are particularly important for data-aware Process Mining and simulation approaches.

TRACE A trace is the chronologically ordered sequence of all events belonging to a specific case. For a truck visit, a trace may appear as follows:

OCR Gate Entry → Gate In → Yard Arrival → Container Delivery → Container
Pick-Up → Gate Out

A trace therefore describes the actual execution path followed by a single process instance.

2.2.3 EVENT LOGS

An event log is the central data structure used in Process Mining. It consists of a collection of events that are grouped into cases and ordered according to their timestamps. At a minimum, each event in an event log contains three essential attributes:

- Case Identifier,
- Activity,
- Timestamp (at least one temporal ordering attribute).

For simulation-oriented process mining, richer event information is required:

- Start Timestamp,
- Complete Timestamp,
- Resource,
- Contextual attributes (e.g. container type, yard utilisation).

The CTB input log used in this thesis provides start and completion timestamps required for execution-time modelling, while activity enablement is reconstructed during process-model alignment by the simulation-discovery framework (Section 4.4).

Additional attributes are usually included to enrich the event data with operational context. Table 2.1 illustrates a simplified event log.

For data exchange between Process Mining tools, event logs are commonly stored in standardized formats. The most widely adopted standard is the eXtensible Event Stream (XES) format. XES was specifically developed for Process Mining applications and enables the storage

Table 2.1: Simplified Event Log Example

Case ID	Activity	Timestamp	Yard Block
T001	Gate In	09:13:04	-
T001	Yard Arrival	09:21:46	Bo4
T001	Gate Out	09:42:18	-
T002	Gate In	09:17:12	-
T002	Yard Arrival	09:30:51	Bo8
T002	Gate Out	09:58:33	-

of events, cases, timestamps, and additional attributes in a structured and tool-independent way. Event logs provide the link between operational information systems and Process Mining techniques. By organizing recorded events into cases and traces, they transform raw operational data into a representation that makes process discovery, conformance checking, and performance analysis possible.

2.3 PROCESS MINING FOUNDATIONS

2.3.1 DEFINITIONS

Organizations increasingly collect large volumes of operational data through enterprise systems such as Enterprise Resource Planning (ERP), Customer Relationship Management (CRM), and Workflow Management Systems (WfMS). Various analytical disciplines have emerged to exploit these data sources. Among the most relevant are Business Intelligence, Data Mining, and Process Mining. Business Intelligence (BI) focuses primarily on descriptive analytics and the monitoring of organizational performance through dashboards, reports, and key performance indicators (KPIs). The goal of BI is to provide decision makers with aggregated information about past and current business performance [1]. Data Mining aims to identify patterns, correlations, classifications, or predictive relationships within large datasets. Traditional data-mining approaches treat observations largely as independent records and often disregard the ordering and dependencies between events [1]. Process Mining bridges the gap between traditional model-based process analysis and data-driven analytics. It exploits event data recorded by information systems to discover, monitor, and improve business processes. Instead of focusing on isolated records, Process Mining explicitly considers the sequence of events that constitute process executions. Van der Aalst defines Process Mining as a discipline that aims to improve

operational processes through the systematic use of event data and process models [1]. A central characteristic of Process Mining is that it combines process models with observed execution data. This enables analysts to discover actual process behaviour, compare observed executions with expected behaviour, identify bottlenecks, and derive insights for process improvement [1].

PROCESS PERSPECTIVES

Business processes can be analysed from different perspectives. Early Process Mining research distinguished four primary perspectives that together provide a comprehensive view of process execution [6].

CONTROL-FLOW PERSPECTIVE The control-flow perspective describes the ordering relations between activities and captures how process instances progress through a process. It focuses on questions such as which activities occur, in which order, and which alternative execution paths exist.

TIME PERSPECTIVE The time perspective focuses on temporal process characteristics such as activity durations, waiting times, throughput times, and bottlenecks. The analysis of timestamps contained in event logs enables the identification of performance issues and delays.

RESOURCE PERSPECTIVE The resource perspective investigates the organizational dimension of a process. It considers which resources execute activities, how work is distributed among employees, and how organizational structures influence process behaviour.

DATA PERSPECTIVE The data perspective analyzes process-related attributes and case characteristics. Examples include customer types, product categories, order volumes, or risk classes. Such attributes often influence process decisions and execution paths [6]. Together, these perspectives provide a multidimensional view of business processes and form the foundation of advanced process analysis techniques.

PROCESS DISCOVERY

Process discovery is the most prominent branch of Process Mining. Its objective is to automatically construct a process model solely from recorded event data without prior process knowledge [1]. The input to process discovery is an event log containing historical process executions. The output is a process model that aims to reproduce the observed behaviour. Depending on the chosen discovery algorithm, the resulting model may be represented as a Petri net, process

tree, BPMN model, or directly-follows graph. Over the years, numerous discovery algorithms have been proposed. Three classical examples are:

- Alpha Miner: One of the earliest process discovery algorithms, capable of identifying causal dependencies and parallel behaviour from event logs.
- Heuristic Miner: An extension of Alpha Miner that incorporates frequency information and is more robust towards noise and infrequent behaviour.
- Inductive Miner: A modern discovery algorithm that recursively decomposes event logs using sequence, exclusive-choice, parallel and loop cuts and guarantees a sound block-structured result. Its treatment of infrequent behaviour and its balance between fitness, precision, generalisation and simplicity make it a strong general-purpose discovery method [21].

Process discovery enables organizations to obtain an objective representation of how their processes are actually executed rather than how they are presumed to operate [1].

PREFIX TREES AND THE OBSERVED TRACE LANGUAGE. A prefix tree, or *trie*, is a rooted tree representation of a set of sequences [22]. Let an event log L be a multiset of traces over activities $\mathcal{A}$. The trie has one node for every observed prefix

$$\text{Pref}(L) = \{\varepsilon\} \cup \{\langle a_1, \dots, a_k \rangle \mid \exists \langle a_1, \dots, a_n \rangle \in L, 1 \leq k \leq n\}, \quad (2.1)$$

where ε denotes the empty prefix. An edge labelled a connects prefix p to $p \cdot \langle a \rangle$ whenever that continuation occurs in the log. Thus, traces with the same history share exactly one path up to their first different activity; nodes and edges may additionally be annotated with occurrence frequencies or data distributions. PRETSA uses precisely this property in process mining: each node groups events of the same activity whose traces share the same activity prefix [23].

Without pruning or smoothing, a prefix trie accepts exactly the observed sequence variants. It therefore maximises replay fitness and precision on its construction log, but it does not infer unseen combinations, loops or concurrency. An Inductive-Miner model makes the opposite trade-off: its block-structured cuts deliberately abstract from individual variants and can admit behaviour not present verbatim in the log [21]. Neither representation is universally superior. The appropriate choice depends on whether the application requires controlled generalisation or a strict, auditable trace language.

Figure 2.1 illustrates the two most common output representations of a modern discovery algorithm on the same event log. The process tree in Figure 2.1a decomposes the process into hierarchically nested operators (*seq* for sequence, *xor* for exclusive choice, *and* for concurrency, *loop* for cyclic behaviour). The BPMN model in Figure 2.1b is the industry-standard graphical notation for the same behaviour and is derived from the process tree by mapping each operator to a BPMN construct (exclusive gateway, parallel gateway, task). Petri-net representations of the same behaviour are structurally equivalent but use a different graphical vocabulary (places and transitions); a concrete Petri-net example on the CTB event log is discussed in Section 5.2.

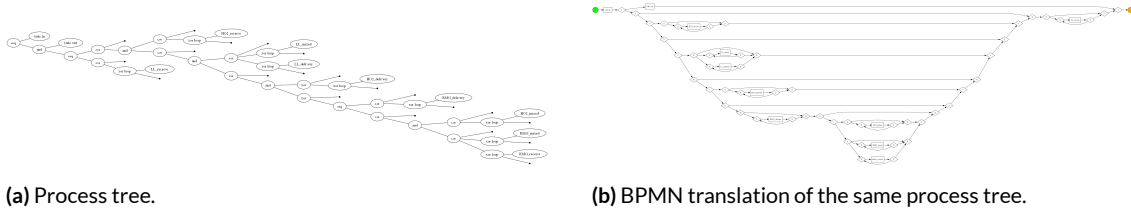


Figure 2.1: Two representations of the same discovered control flow (inductive miner on the CTB training log). The process tree exposes the hierarchical operator structure; the BPMN translation preserves the same behaviour using the industry-standard graphical vocabulary. Both models describe an identical set of allowed traces and are used interchangeably in the process mining literature [1].

CONFORMANCE CHECKING

While process discovery focuses on constructing process models, conformance checking focuses on comparing observed process executions with an existing process model [24]. The objective is to identify deviations between the model and reality. Such deviations may indicate compliance violations, process inefficiencies, exceptional cases, or outdated process documentation. The quality of discovered and analyzed process models is commonly evaluated using four criteria [24, 1]:

- **Fitness:** Measures the extent to which observed process executions can be replayed by the process model.
- **Precision:** Evaluates whether the model avoids allowing excessive behaviour that was not observed in reality.
- **Generalization:** Assesses whether the model captures underlying process behaviour beyond the specific instances present in the event log.

- **Simplicity:** Favours models that explain observed behaviour with minimal complexity while remaining understandable.

These criteria provide a balanced framework for evaluating process model quality and form the basis of many process-mining evaluation approaches.

A*-BASED OPTIMAL ALIGNMENT. An alignment relates an observed trace to an executable run of a process model. Formally, it is a sequence of three move types: a *synchronous move* consumes an event and fires a transition with the same visible label; a *log move* consumes an event without a corresponding model transition; and a *model move* fires a model transition without consuming an observed event. A cost function normally assigns zero cost to synchronous moves and positive, configurable costs to deviations. An optimal alignment λ^* is therefore

$$\lambda^* = \arg \min_{\lambda \in \Lambda(\sigma, N)} c(\lambda), \quad (2.2)$$

where $\Lambda(\sigma, N)$ is the set of legal alignments between trace σ and process model N [24, 25].

The qualifier “A*-based” refers to the search procedure, not to a different kind of alignment. A* explores the product state space of the current trace position and the current model state (for a Petri net, its marking). It orders candidates by $f = g + h$, where g is accumulated move cost and h is an admissible lower bound on the remaining cost. With an admissible heuristic, the first complete solution is cost-optimal [25]. The resulting event–transition correspondence supports conformance diagnostics and timestamp replay for performance analysis [24]; it does not turn a model-derived enablement instant into an independently observed timestamp.

PROCESS ENHANCEMENT

The Process Mining literature commonly distinguishes three major categories of process mining techniques: process discovery, conformance checking, and process enhancement [1]. Whereas discovery creates a process model from event logs and conformance checking compares observed behaviour with existing models, process enhancement aims to improve existing process models using information extracted from event data. Enhancement may take different forms. For example, performance information such as waiting times or activity durations can be projected onto a process model. Likewise, resource assignments, bottlenecks, and workload information can be incorporated to create richer process representations. Process enhancement therefore enables organizations not only to understand how processes operate, but also to identify opportunities for process improvement and optimization [1].

2.4 BUSINESS PROCESS SIMULATION

2.4.1 FUNDAMENTALS OF BUSINESS PROCESS SIMULATION

Business Process Simulation (BPS) is a widely established analytical technique used to reproduce and analyze the dynamic behavior of business processes within a virtual environment. Rather than experimenting directly on operational systems, organizations build executable models that represent essential process characteristics and repeatedly execute these models under controlled conditions. This allows process behaviour to be observed without interfering with live operations [26]. Van der Aalst defines simulation as a flexible approach for analysing business processes and evaluating alternative operational scenarios. By recreating process executions under varying conditions, simulation can estimate process performance indicators such as throughput times, waiting times, resource utilization, service levels, and operational costs before changes are implemented in practice [26]. The primary purpose of simulation is decision support. Business processes are often characterized by uncertainty, dynamic resource interactions, and complex dependencies that make analytical evaluation difficult. Simulation provides a safe and cost-efficient environment for assessing operational changes while avoiding the risks associated with real-world experimentation [26]. One of the most important applications of simulation is *what-if analysis*. What-if analyses evaluate how process performance changes under alternative assumptions or operational configurations. Typical examples include increasing the number of available resources, modifying routing rules, changing workload levels, or introducing new operational policies. By comparing simulation outcomes across different scenarios, decision makers can assess the expected consequences of proposed changes before implementation [26]. Because of its ability to represent dynamic process behaviour and operational dependencies, simulation has become an established decision-support tool in fields such as manufacturing, healthcare, logistics, supply-chain management, and business process management. More recently, Process Mining research has increasingly integrated simulation with event data, enabling simulation models to be discovered and calibrated directly from recorded process executions [6, 14].

2.4.2 BUSINESS PROCESS SIMULATION MODELS

A simulation model is a simplified representation of a real-world process. Although specific modeling approaches vary, most business process simulation models consist of several funda-

mental perspectives that jointly determine process behaviour during simulation runs [26, 6].

CONTROL FLOW

The control-flow perspective describes the behavioural structure of the process. It determines which activities exist, in which order they may occur, and which alternative execution paths are possible. In simulation models, the control-flow model serves as the structural backbone that governs process execution. Common representations include Petri nets, BPMN models, process trees, and workflow graphs [1, 26].

ARRIVAL PROCESS

The arrival process defines how new process instances enter the system. In business processes, this typically corresponds to customer requests, orders, applications, service tickets, or other incoming work items. Arrival behaviour is commonly represented through statistical distributions that describe the time between consecutive process arrivals. Since workload levels directly influence waiting times and resource utilization, realistic arrival modelling is essential for meaningful simulation results [26, 6].

RESOURCES

Resources represent the entities that execute process activities. Depending on the application domain, resources may correspond to employees, machines, vehicles, cranes, terminal equipment, or information systems. Resource models typically include information about availability, capacities, execution capabilities, working schedules, and allocation policies. Resource constraints are often a major determinant of process performance because multiple process instances compete for limited resource capacity [26].

ACTIVITY DURATIONS

The timing perspective describes how long activities require for execution. Typical simulation models distinguish between processing times, waiting times, and overall case durations. Activity durations are frequently represented through probability distributions derived from historical observations. These temporal characteristics largely determine process performance and throughput behaviour [26, 6].

DECISION LOGIC

Many business processes contain decision points where alternative execution paths may be selected. Decision logic determines how these routing decisions are made during simulation. Traditional simulation models commonly employ static branching probabilities that specify how often a particular path is chosen. More advanced approaches model decisions as functions of contextual information, process attributes, or operational conditions [6, 8]. Together, control flow, arrivals, resources, timing information, and decision logic define the behaviour of a simulation model. These perspectives form the foundation of both traditional manually developed simulation models and modern simulation-discovery approaches discussed later in this chapter.

2.4.3 SIMULATION-BASED DECISION SUPPORT

Container terminals operate in highly dynamic environments characterized by fluctuating demand, limited resource capacity, and strong interdependencies between operational processes. Managers must continuously make decisions regarding equipment allocation, workforce planning, storage strategies, truck appointment policies, and capacity utilization while maintaining uninterrupted terminal operations. Direct experimentation in a live terminal environment is often impractical because operational changes may negatively affect service quality, throughput, or customer satisfaction. As a consequence, simulation has become one of the most widely applied decision-support approaches in container terminal research [27, 17, 18]. By providing a virtual representation of terminal operations, simulation enables decision makers to evaluate operational alternatives before implementation. Typical applications include:

- evaluation of truck appointment policies,
- resource and workforce planning,
- capacity planning and investment decisions,
- yard layout and storage strategies,
- yard crane allocation policies,
- congestion analysis and bottleneck identification,
- equipment scheduling and maintenance planning.

A central advantage of simulation is its ability to support what-if analyses. Alternative operational scenarios can be evaluated under identical conditions, allowing direct comparison of performance measures such as truck turnaround times, resource utilization, throughput, waiting times, and service levels. This capability is particularly valuable in container terminals because small operational changes frequently generate system-wide effects due to the strong interdependencies between gate operations, yard processes, and vessel handling activities [3, 27, 17]. For truck operations, simulation has been extensively applied to study the effects of Truck Appointment Systems (TAS), gate capacities, yard utilization, and equipment allocation strategies. Such analyses support terminal operators in balancing efficiency improvements with operational robustness under uncertain workloads and changing operating conditions [5, 16].

2.4.4 SIMULATION MODEL VERIFICATION, CALIBRATION AND VALIDATION

The usefulness of a simulation model depends not only on its structure but also on its ability to represent real-world behaviour with sufficient accuracy. Consequently, simulation projects typically include activities related to verification, calibration, and validation [26].

VERIFICATION Verification addresses the question:

“Was the simulation model implemented correctly?”

The objective of verification is to ensure that the executable simulation model correctly reflects the conceptual model and contains no implementation errors. Verification therefore focuses on the internal consistency of the model rather than its correspondence with reality [26].

CALIBRATION Calibration concerns the estimation and adjustment of simulation parameters so that the model accurately reflects observed operational conditions. Typical calibration tasks include fitting arrival distributions, activity durations, resource capacities, and routing probabilities using historical observations. Calibration therefore links the conceptual model with empirical process data [6, 14].

VALIDATION Validation addresses the question:

“Does the simulation model adequately represent the real system?”

Validation evaluates whether simulation outputs correspond sufficiently well with observed process behavior. This often involves comparing simulated and real performance measures such as throughput times, waiting times, resource utilization, or process paths. A validated model provides confidence that simulation results can be used for decision support and process analysis [26, 6]. Historically, these activities have been performed manually as part of traditional simulation studies. However, the emergence of simulation discovery and Process Mining has increasingly shifted verification, calibration, and validation towards data-driven approaches. Automatically discovered simulation models can use historical event data both for parameter estimation and for systematic comparisons between simulated and observed process behavior [6, 14, 28].

2.5 SIMULATION DISCOVERY AND DATA-AWARE SIMULATION

Here you should recognise the components from "Business Process Simulation"

The development of simulation models traditionally relies on a lot of manual modeling efforts and substantial domain expertise. Process analysts and simulation experts typically construct process models, resource configurations, routing probabilities, and performance parameters based on interviews, observations, and historical reports. [SOURCE] This approach is intuitive and expertise based, but often costly, time-intensive, and prone to subjective assumptions. To reduce modeling effort and alignment with observed process behavior, Process Mining research focuses on the automatic discovery of simulation models directly from historic process data in the form of event log. This research stream is commonly referred to as *simulation discovery*.

2.5.1 FROM MANUAL SIMULATION TO SIMULATION DISCOVERY

Traditional simulation projects require process experts to manually define control-flow structures, estimated activity durations, planned resource assignments, and routing probabilities. Consequently, the resulting simulation models often reflect how processes are expected to operate rather than how they are actually executed in practice [6]. Furthermore, simulation models require continuous maintenance and recalibration to remain aligned with changing operational conditions. One of the first comprehensive approaches to automatically derive simulation models from event logs was proposed by Rozinat et al. [6]. Their framework com-

combines Process Mining techniques with simulation modelling and automatically discovers multiple simulation perspectives, including process control-flow, activity durations, resource assignments, and branching probabilities. By deriving simulation parameters directly from historical event data, the resulting models can more accurately reflect real process executions while reducing modelling effort. Building upon this idea, subsequent research increasingly automated simulation model construction. Camargo et al. [14] introduced a framework for the automated discovery of business process simulation models from event logs. Their approach systematically extracts simulation parameters and integrates them into executable simulation models. More recent work further extends simulation discovery through machine learning and deep learning techniques, enabling simulation models to capture complex behavioural patterns directly from operational data. While these approaches often improve predictive accuracy, they may also reduce transparency and configurability compared to traditional simulation models. As a result, simulation discovery has emerged as an important research area at the intersection of Process Mining and Business Process Simulation, aiming to automatically generate simulation models that faithfully represent observed process behaviour while minimizing manual modelling effort.

2.5.2 DISCOVERY OF SIMULATION PARAMETERS

Simulation models consist of multiple perspectives that must be discovered and calibrated from historical event data. Modern simulation discovery approaches therefore decompose the discovery problem into several parameter categories.

CONTROL-FLOW DISCOVERY

Control-flow discovery aims to reconstruct the behavioural structure of a process from event logs. Process discovery algorithms such as the Inductive Miner derive formal process models, commonly represented as Petri nets, BPMN models, or process trees. The discovered process model serves as the behavioural backbone of the simulation model and defines the possible execution paths that process instances can follow [1, 6].

ARRIVAL DISCOVERY

Arrival discovery focuses on modelling how new process instances enter the system. In simulation models, this typically requires estimating the inter-arrival times between subsequently arriving cases. Arrival patterns are commonly derived from historical timestamps and represented

through statistical distributions or predictive models. Accurate arrival modelling is essential because workload dynamics strongly influence waiting times, queue lengths, and resource utilization [6, 12].

RESOURCE DISCOVERY

Resource discovery concerns the identification of the resources involved in process execution and the assignment of activities to these resources. Resource-aware simulation models additionally require information about organizational structures, resource calendars, availability patterns, and allocation policies. Modern simulation-discovery approaches increasingly model resource behaviour as a context-dependent phenomenon instead of assuming static activity-resource mappings [29, 12].

TIMING DISCOVERY

Timing discovery aims to capture the temporal behaviour of process executions. Typical parameters include activity execution times, waiting times, service times, and overall throughput times. Early approaches estimate these parameters using fitted statistical distributions, whereas more recent approaches employ machine-learning techniques to model context-dependent temporal behaviour. Timing information is crucial for reproducing realistic process performance and resource interactions during simulation [6, 14, 12].

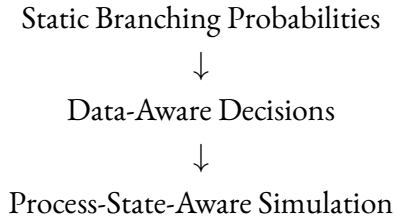
DECISION LOGIC DISCOVERY

Many business processes contain decision points at which alternative execution paths may be chosen. Traditionally, these decisions are modelled using branching probabilities derived from historical frequencies. Such probabilities reflect how often a specific path was observed in the event log. However, routing decisions in real processes are often influenced by process attributes, execution history, resource conditions, and operational context. Consequently, recent simulation-discovery approaches increasingly focus on discovering data-dependent decision rules that dynamically adapt simulation behaviour to the current process context [8, 9, 12].

2.5.3 DATA-DRIVEN AND DATA-AWARE SIMULATION [ONLY CASE LEVEL?]

Early simulation-discovery approaches typically represent decision-making behaviour using static branching probabilities. At a decision point, the probability of selecting a particular execution path is estimated from the observed frequency of that path in the event log. While such prob-

abilities provide a simple representation of process behaviour, they cannot explain why different decisions occur under different circumstances [6]. To overcome these limitations, recent research has introduced data-aware simulation approaches. Instead of relying solely on global routing frequencies, process decisions are conditioned on contextual information describing the process instance. Mannhardt et al. [8] propose the concept of data-aware stochastic process models, where transition probabilities depend on process attributes rather than being represented as fixed probabilities. This allows simulation models to differentiate between cases with different characteristics and reproduce behaviour that depends on process data. Building upon this idea, de Leoni, Vinci, Leemans, and Mannhardt [9] introduce the concept of *process states*. A process state represents a data-aware abstraction of a process instance and may include process attributes, activity execution history, and other contextual information accumulated throughout process execution. Instead of assigning a single probability to an enabled transition, activity-firing probabilities become dependent on the current process state. The authors demonstrate that simulation models based on process-state-aware decision rules achieve higher behavioural accuracy than traditional frequency-based branching probabilities. This transition can be summarized as follows:



The most recent developments extend this idea beyond control-flow decisions. In the Prosit framework, Vinci et al. [12] propose Decision-Aware Business Process Simulation models that discover control-flow decisions, execution times, waiting times, and resource behaviour using probabilistic decision trees. These models condition simulation parameters on contextual features such as trace attributes, process history, resource assignments, and workload-related information. An important distinction must be made between trace-level attributes and dynamically changing process-state information. The current Prosit framework primarily utilizes trace attributes, trace history, resource information, and workload-related features for simulation parameter discovery [12]. Arbitrary event-level contextual attributes that change continuously during process execution are not explicitly included in the published feature definitions. Nevertheless, the process-state perspective introduced by recent research suggests that richer operational context information may further improve simulation realism, particularly in complex

operational environments such as container terminals. The evolution from frequency-based branching probabilities towards context-dependent and process-state-aware simulation models represents one of the most significant developments in simulation discovery research and provides the foundation for modern explainable and white-box simulation approaches [9, 12].

CONFIGURABLE EXPRESSIVENESS IN PROSiT. At the API level, ProSiT exposes the expressiveness of the discovered simulator through two configuration parameters that will be used to design the three experimental configurations in Section 4.4:

- `max_depth_tree` controls the maximum depth of the probabilistic decision trees fitted for each simulation dimension (arrival time, execution time, waiting time, routing, resource selection). Setting `max_depth_tree=0` disables the trees entirely and reduces the framework to a distribution-based simulator that samples every parameter from a single fitted distribution. Higher values allow deeper conditional partitions of the case-attribute space, at the cost of overfitting risk that ProSiT counteracts through a cross-validated leaf-size grid and a fall-back to the unconditional distribution when no tree beats the marginal baseline on held-out CRPS.
- `use_workload_features` decides whether the waiting-time and resource-selection models receive two additional queue-oriented features (current workload and queue length at the enabling time of the event). When enabled, the fitted trees can condition on the local queueing state and not only on the case attributes.

The interplay of these two knobs defines a natural comparison ladder between a distribution-only simulator, a context-aware simulator and a context-plus-workload-aware simulator. This ladder is exploited in Chapters 4 and 5 to test whether the additional modelling machinery of ProSiT translates into higher held-out fidelity on the CTB truck process.

2.6 EXPLAINABLE AND WHITE-BOX SIMULATION

This chapter is suppose to explain why explainability is becoming a major research problem in simulation discovery, and then positioning Prosit as the current state-of-the-art answer to that problem.

2.6.1 EXPLAINABILITY IN SIMULATION-BASED DECISION SUPPORT

Business Process Simulation is increasingly used to support operational and strategic decision making. Simulation results often form the basis for resource-allocation decisions, process redesign initiatives, capacity planning, and investment evaluations. Consequently, decision makers must be able to understand why a simulation model produces a particular outcome and how simulation behaviour changes when process parameters are modified. Traditional Business Process Simulation models are inherently transparent. Their control-flow structures, routing probabilities, resource allocations, and timing assumptions are manually specified and can therefore be inspected and modified by domain experts. As a result, simulation users can directly understand the assumptions underlying the generated simulation results [26]. Recent advances in simulation discovery increasingly employ machine-learning and deep-learning techniques to improve simulation accuracy. These approaches learn behavioural patterns directly from historical event data and often outperform manually calibrated models in terms of predictive capabilities [14, 30]. However, the resulting models frequently behave as black-box systems. While they can generate accurate predictions, the internal decision mechanisms that govern simulation behaviour are not readily interpretable by human analysts. This creates a tension between predictive accuracy and explainability. Black-box simulation models may successfully reproduce historical process behaviour while simultaneously limiting the analyst's ability to understand why particular routing decisions, waiting times, or resource allocations are generated. Consequently, the simulation model becomes difficult to inspect, configure, validate, and trust for operational decision support [11]. The explainability problem becomes particularly relevant in what-if analyses. Business Process Simulation is not only intended to reproduce observed behaviour but also to evaluate hypothetical future scenarios. Such analyses require analysts to intentionally modify process parameters, resource settings, or decision rules. In black-box simulation approaches, the underlying behavioural logic is often inaccessible, making it difficult to assess whether the simulated reactions to process changes remain realistic and trustworthy [11]. To address these limitations, recent research has started investigating explainable simulation models that combine the predictive capabilities of machine-learning approaches with transparent and interpretable decision logic. The goal is not only to reproduce process behaviour accurately but also to expose the reasoning mechanisms that determine simulation outcomes. Explainability therefore becomes an important prerequisite for trustworthiness, configurability, and practical adoption of data-driven simulation models.

2.6.2 PROBABILISTIC WHITE-BOX SIMULATION MODELS

The concept of white-box simulation emerged as a response to the increasing reliance on opaque machine-learning models for simulation parameter discovery. White-box simulation models aim to make behavioural decisions explicitly visible and interpretable while maintaining a high degree of simulation accuracy. A key motivation behind white-box simulation is that business analysts must be able to inspect, understand, and modify the simulation behaviour that drives decision-support outcomes. Unlike black-box predictors, white-box approaches expose the discovered decision rules and allow analysts to evaluate the causal relationships that influence simulation parameters. An important step towards explainable simulation is represented by the Runtime Integration of Machine Learning and Simulation (RIMS) framework proposed by Meneghello et al. [11]. RIMS combines traditional discrete-event simulation with predictive machine-learning models at runtime. The authors explicitly argue that purely deep-learning-based simulation approaches suffer from limited explainability and restricted support for what-if analyses. By maintaining an explicit simulation model while integrating learned predictions during execution, RIMS seeks to combine simulation transparency with data-driven prediction capabilities [11]. More recently, Vinci et al. introduced the ProSiT framework (*PROcess Simulation Tool*), which represents one of the first dedicated white-box simulation-discovery approaches [12]. ProSiT extends data-aware simulation discovery by replacing opaque prediction mechanisms with probabilistic decision trees. These trees are used to represent multiple simulation dimensions including:

- control-flow decisions,
- activity execution times,
- waiting times,
- resource assignment behaviour,
- arrival processes.

The central idea of ProSiT is that simulation behaviour should remain directly interpretable. Instead of producing predictions through hidden neural-network representations, behavioural rules are exposed explicitly in the form of decision-tree structures. Analysts can therefore inspect, understand, and modify the conditions influencing simulation behaviour [12]. An additional contribution of ProSiT is the integration of uncertainty through probabilistic deci-

sion trees. Traditional decision trees often generate deterministic predictions. In contrast, probabilistic decision trees capture distributions of possible outcomes and thereby preserve the stochastic nature required for realistic process simulation [12]. This is particularly important because simulation models aim not only to reproduce average behaviour but also the variability observed in real operational environments. Experimental evaluations presented by Vinci et al. indicate that probabilistic white-box models can achieve simulation accuracy comparable to, and in some cases exceeding, alternative black-box approaches while simultaneously improving interpretability and configurability [12]. Furthermore, the framework enables process analysts to directly edit discovered rules and evaluate modified decision scenarios without retraining complex machine-learning models. Despite these promising results, research on white-box simulation remains at an early stage. Current evidence is largely limited to benchmark datasets and business-process environments investigated by the authors themselves. Large-scale empirical evaluations across different domains remain scarce. In particular, little evidence currently exists regarding the behaviour of white-box simulation models in highly dynamic cyber-physical systems such as container terminals. Moreover, explainability itself remains difficult to quantify. While decision trees are generally considered more transparent than neural networks, there is currently no universally accepted framework for measuring the practical interpretability of simulation-discovery models. Existing evaluations primarily focus on simulation accuracy, leaving questions related to user trust, decision-support quality, and operational usefulness largely unexplored. Consequently, white-box simulation should not yet be regarded as a mature solution to the explainability challenge. Rather, it represents an emerging research direction that attempts to balance three competing objectives:

1. simulation accuracy,
2. simulation explainability,
3. simulation configurability.

The ProSiT framework currently represents one of the most comprehensive realizations of this vision and serves as the methodological foundation for the simulation approach investigated in this thesis [12].

2.7 SIMULATION QUALITY, VALIDATION/ASSESSMENT, AND TRUSTWORTHINESS

This part is suppose to answer the question: How do we know whether these automatically discovered simulation models are actually good enough to support decisions?

As simulation models increasingly become automatically generated from event data, questions regarding their quality, reliability, and practical usefulness become increasingly important. While traditional simulation projects rely heavily on expert validation and manual calibration, simulation-discovery approaches automatically derive model structures and parameters directly from event logs. Consequently, the assessment of simulation quality has evolved into an active research area at the intersection of Process Mining and Business Process Simulation. Ultimately, the objective of a simulation model is not merely to reproduce historical data, but to support operational decision making. Therefore, simulation quality must be evaluated from multiple perspectives, including behavioural realism, temporal accuracy, resource representation, and decision-support capability.

2.7.1 DIMENSIONS OF SIMULATION QUALITY

Simulation models are multi-dimensional representations of real-world processes. Consequently, simulation quality cannot be reduced to a single performance indicator. Instead, different aspects of process behaviour must be evaluated separately.

CONTROL-FLOW QUALITY The control-flow perspective evaluates whether the simulation model reproduces the behavioural structure observed in reality. A high-quality model should generate process variants and activity sequences that resemble the execution behaviour recorded in the event log. Deviations may indicate missing process paths, unrealistic routing behaviour, or overgeneralized models [6, 31].

TEMPORAL QUALITY Temporal quality measures how accurately the simulation reproduces timing-related process characteristics such as activity durations, waiting times, throughput times, and overall case durations. Since decision-support applications frequently focus on performance improvement, temporal realism is often considered a critical component of simulation quality [26, 31].

RESOURCE QUALITY Resource quality evaluates how accurately simulation models represent resource behaviour, including resource assignments, workloads, utilization patterns, and capacity constraints. Resource-related inaccuracies can significantly affect simulation outcomes because waiting times and bottlenecks are often produced by resource contention rather than control-flow decisions alone [26, 12].

DECISION-SUPPORT CAPABILITY A technically accurate simulation model does not automatically provide useful decision support. Simulation results must remain sufficiently realistic under changed conditions and alternative scenarios. Consequently, simulation quality also includes the ability of a model to support meaningful what-if analyses and generate trustworthy recommendations for operational decision making [31, 11].

2.7.2 SIMULATION VALIDATION

Traditional simulation validation typically compares selected performance indicators between observed and simulated systems. However, modern simulation-discovery approaches require more comprehensive evaluation techniques because behavioural realism, timing characteristics, and workload dynamics must all be assessed simultaneously. Recent Process Mining research therefore introduced a variety of similarity metrics designed to compare event logs generated by simulation models with real-world event logs [31].

CONTROL-FLOW SIMILARITY METRICS

Control-flow similarity focuses on comparing the behavioural structure of simulated and observed process executions. One of the most widely used techniques is the Earth Mover's Distance (EMD), originally introduced for stochastic conformance checking by Leemans et al. [32]. EMD measures how much probability mass must be moved between two trace distributions to transform one distribution into another. A lower EMD indicates stronger behavioural similarity. Building upon this idea, simulation-specific evaluation frameworks frequently employ Control-Flow Log Distance (CFLD) metrics. CFLD compares the frequencies of process variants generated by a simulation model against the variants observed in reality. This enables the assessment of behavioural realism at the process level [31]. Another common family of control-flow measures is based on N-Gram analysis. Rather than comparing complete traces, N-Gram methods compare local activity patterns consisting of consecutive activity sequences of length n . This allows behavioural similarities to be identified even when complete process variants

differ [31].

TEMPORAL SIMILARITY METRICS

Control-flow similarity alone is insufficient because simulation models are frequently used to predict temporal process performance. Case Throughput Distance (CTD) compares throughput time distributions between simulated and real event logs. The metric evaluates how accurately a simulation reproduces the overall duration of process instances [31]. Absolute Error Distance (AED) and Relative Error Distance (RED) quantify the discrepancy between observed and simulated temporal characteristics. These metrics are commonly used to assess waiting times, service times, and throughput times at different levels of aggregation [31]. Temporal metrics are particularly relevant in operational environments where process performance directly affects service quality. In container terminal operations, for example, accurate prediction of truck turnaround times and waiting times represents a central requirement for practical simulation-based decision support.

ARRIVAL AND CONGESTION METRICS

In many operational systems, process performance is strongly influenced by workload levels and arrival dynamics. Consequently, simulation accuracy must also be evaluated with respect to congestion behaviour and workload generation. The Case Arrival Rate (CAR) metric compares the arrival patterns of process instances between simulated and observed event logs. Accurate arrival modelling is essential because unrealistic workload generation can distort resource utilization, queue lengths, and waiting times [31]. For container terminal operations, arrival behaviour is particularly important because truck arrivals and yard workloads directly influence congestion levels throughout the terminal. A simulation model that accurately reproduces individual process traces but fails to replicate realistic truck arrival dynamics may still generate misleading decision-support results.

WASSERSTEIN DISTANCE

Many similarity metrics used in simulation validation are based on the Wasserstein Distance, which originated in optimal transport theory. The Wasserstein Distance measures the minimum effort required to transform one probability distribution into another. Intuitively, it can be interpreted as the amount of work required to relocate probability mass between distributions. Earth Mover's Distance represents a discrete formulation of the Wasserstein Distance and is particularly well suited for comparing trace distributions and temporal distributions in

Process Mining applications [32]. Due to its ability to compare complete distributions rather than only summary statistics, Wasserstein-based metrics have become a central component of modern simulation validation frameworks.

KOLMOGOROV–SMIRNOV STATISTIC

The two-sample Kolmogorov–Smirnov (KS) statistic is a complementary distributional distance that measures the maximum vertical gap between two empirical cumulative distribution functions F_{sim} and F_{real} :

$$KS = \sup_x |F_{\text{sim}}(x) - F_{\text{real}}(x)|. \quad (2.3)$$

The KS statistic is unitless, bounded in $[0, 1]$ and can be paired with a two-sample significance test to decide whether the sim/real samples could plausibly originate from the same underlying distribution. In practical simulation-validation work, the KS statistic is often reported *alongside* the Wasserstein distance: Wasserstein carries the operational units (minutes for a duration) while the KS statistic normalises across scales and makes distributional similarity comparable between activities of very different mean duration.

MONTÉ-CARLO REPLICATION AND CONFIDENCE INTERVALS

Even a fully specified discovered simulator produces a different sample every time it is run, because every parameter is sampled from a fitted distribution. Reporting the outcome of a single simulation run is therefore not sufficient to conclude that one configuration is more faithful than another: the observed distance between two configurations may lie within the intrinsic Monte-Carlo variance of the simulator itself. A principled evaluation therefore repeats every simulation N times with independent random seeds and aggregates the resulting distances into a confidence interval, typically a Student-t interval on the mean or a bootstrap percentile interval. This practice is standard in discrete-event simulation [26] but is often omitted in reports of simulation-discovery evaluations. Chapter 4 therefore specifies an explicit multi-seed replication protocol so that every fidelity claim in the results chapter can be qualified by its 95 % confidence interval.

2.7.3 TRUSTWORTHINESS OF AUTOMATICALLY DISCOVERED SIMULATION MODELS

As simulation models become increasingly data-driven and automatically generated, simulation quality alone is no longer sufficient. Decision makers must also be able to trust the simulation model and understand its limitations. Recent research therefore increasingly focuses on the trustworthiness of simulation-discovery approaches. Trustworthiness encompasses multiple dimensions, including behavioural accuracy, robustness, explainability, reproducibility, and decision-support reliability [11, 12]. The framework proposed by Chapela-Campa et al. represents one of the first systematic efforts to measure simulation quality across multiple dimensions simultaneously [31]. Their work highlights that simulation accuracy depends on different behavioural perspectives and that no single validation metric is sufficient to judge model quality comprehensively. Similarly, recent healthcare studies using the ProSiT framework demonstrate that improved behavioural realism can positively influence simulation-based resource-allocation decisions [13]. However, these studies remain largely confined to healthcare and administrative business processes. Although promising results have been reported, empirical evidence regarding the robustness and trustworthiness of automatically discovered simulation models remains limited. Most evaluations focus on reproducing historical behaviour rather than assessing simulation performance under substantially changed operational conditions. Consequently, important questions regarding model stability and decision-support reliability remain only partially addressed.

2.7.4 RESEARCH GAP

Container terminals represent highly dynamic and resource-constrained operational environments characterized by complex interactions between truck processes, yard operations, storage systems, and handling equipment. As a result, simulation has become one of the most widely used analytical methods for supporting operational decision making in container terminal management [27, 17, 18]. Despite this extensive use of simulation, most container terminal simulation models continue to be developed manually. Model construction, calibration, and maintenance require substantial modelling effort and domain expertise, limiting scalability and practical applicability in rapidly changing operational environments [6]. Simulation discovery addresses this limitation by automatically generating simulation models from event logs. Over the past decade, significant advances have been achieved in the discovery of simu-

lation parameters, data-aware decision logic, and automated model construction [14, 28, 12]. More recently, data-aware simulation approaches demonstrated that contextual information and process-state representations can substantially improve simulation accuracy compared to traditional branching probabilities [8, 9]. In parallel, white-box simulation approaches such as ProSiT introduced interpretable probabilistic decision models that aim to improve transparency and configurability while maintaining strong predictive performance [12]. However, existing empirical evaluations have primarily been conducted in healthcare, administrative, and generic business-process environments. To date, there is limited evidence regarding the applicability and behaviour of simulation-discovery approaches within large-scale container terminal operations. Furthermore, the increasing complexity of data-aware and decision-aware simulation models raises unresolved questions concerning:

- the robustness of automatically discovered simulation models,
- their trustworthiness for operational decision support,
- their ability to reproduce congestion effects and workload dynamics,
- the practical value of white-box simulation in complex logistics environments.

This thesis does not aim to provide a general answer to these broader research challenges. Rather, it contributes an empirical case study within the context of truck-handling operations at the HHLA Container Terminal Burchardkai (CTB). By evaluating simulation-discovery approaches in a real-world container terminal environment, the thesis provides evidence regarding their applicability, strengths, and limitations in a domain that has received comparatively little attention within the existing simulation-discovery literature.

does this give of: I am not solving trustworthiness of simulation discovery here! I am providing a container-terminal case study that contributes evidence toward that larger research question.

3

Case Study and Data Engineering

This chapter documents the operational context in which the data-aware simulation is discovered and the execution pipeline that turns raw operational records into a simulation-ready event log. The chapter is dedicated to only outline the design, therefore no models are fitted, no hypotheses are tested, and no experimental parameters are chosen yet. Its purpose is to establish the data foundation on which every subsequent claim in Chapters 4 and 5 builds on, so that these later claims and results can be analysed against their exact input.

Section 3.1 introduces the HHLA Container Terminal Burchardkai (CTB) in Hamburg, Germany, the truck-handling scope of the study and the data sources that feed the event-log engineering pipeline. Section 3.2 describes the multi-stage transformation that produces the final event log, including lifecycle-timestamp reconstruction and the derivation of case-level context attributes required by the ProSiT simulation framework [12]. The chapter closes with a summary of the final event-log characteristics that are used as input to the train/test split defined in Section 4.2.

3.1 CASE STUDY ENVIRONMENT

3.1.1 HHLA CONTAINER TERMINAL BURCHARDKAI

The case study's environment is the HHLA Container Terminal Burchardkai (CTB), one of Europe's largest container terminals, located in the Port of Hamburg. CTB handles approximately 2.6 million TEU (Twenty-foot Equivalent Units) annually across 315 hectares [33].

Truck Process Scope: External trucks visit CTB to deliver or receive containers. The process comprises:

1. Gate In: Truck registers at the entry gate.
2. Container handling: Rail-Mounted Gantry cranes (RMGs), Van Carriers (VC) or manual operators (HO₂) load or unload containers at storage blocks.
3. Gate Out: Truck exits the terminal.

The CTB environment in this project has several resource constraints. Located between the waterside and the landside, 22 RMG storage blocks (To6–T27) serve both sides. Operationally, at most three automated gantry cranes can work in parallel at one block. The CTB also has two specialised areas: an empty-container depot (called “MT” or “LL”, from the German *Leerlager*) and the manually operated HO₂ yard area. The terminal gate consists of 16 lanes, split between incoming and outgoing trucks. The number of operated gate lanes varies by day, shift and terminal resource plan.

3.1.2 DATA SOURCES

The operational data used in this thesis originates from several information systems that support day-to-day terminal operations. Historically, the information-system landscape at CTB evolved organically, resulting in a heterogeneous environment in which different systems manage distinct aspects of terminal operations, orchestrated by a Terminal Operating System. The primary data sources for this study are:

- ITS schedule (scheduling system) — records truck visit schedules, including timestamps for each operational step (gate-in time, ready time at a handling point, order fulfilment time, gate-out time).

- Report system — captures stage-level execution data, including gate entry times, exchange-area assignments and container transaction records, bridging yard operations and container master data.
- Yard management system — snapshots every four hours of all yard block capacities and utilisation rates.
- Container master data — container-level details (size, type, hazardous material, dwell time, inbound and outbound carrier).

These systems generate independent data pools with different identifier schemes, requiring the data-integration procedure described in Section 3.2. Historical data from March and April 2026 (8 weeks) was extracted, covering approximately 104,000 truck visits.

Data volume:

- 113,466 container move events,
- 104,000 truck visits,
- 336 yard-state snapshots for (8 weeks $\times$ 7 days $\times$ 6 snapshots per day).

3.2 EVENT LOG ENGINEERING

Event-log engineering is the first stage of the experimental pipeline. The objective is to construct an integrated, process-oriented event log from the raw operational data that satisfies the requirements of both process mining and data-aware simulation while retaining as much operational context as possible. This section describes the pipeline stage by stage.

3.2.1 RAW DATA SOURCES AND PREPARATION

ITS SCHEDULE DATA

The ITS scheduling system provides the core scheduling data for the truck operations reconstructed in this thesis. Each record represents one activity instance within a truck visit and carries the following key attributes:

- FAHRPLAN_UID — unique identifier for the truck visit (case ID).

- LKW_KENNZEICHEN — truck license plate number.
- ANLAUFUNKT_HALTESTELLE — exchange area identifier (e.g. T20, LL, VC).
- FP_ZEITPUNKT_ERSTELLUNG — timestamp when the schedule entry was created (gate-in time).
- ALP_ZEITPUNKT_BEREITMELDUNG — timestamp when the activity became ready for execution (enabled time).
- ALP_ZEITPUNKT_ERFUELLUNG — timestamp when the activity was completed.
- FP_ZEITPUNKT_GATEOUT — timestamp when the truck exited the terminal.

All timestamp fields are parsed into standardised `datetime` objects using `pandas` with the format `%d.%m.%Y %H:%M`. Invalid or missing timestamps are flagged for removal during the cleaning phase.

REPORT SYSTEM DATA

The report system provides complementary stage-level execution data, completing the details for gate-entry events, and it links each container number to the specific truck visit at a yard handling point. Key attributes are:

- `TruckVisitGkey` — unique truck-visit identifier used for cross-system matching.
- `TruckLicenseNbr` — truck license plate.
- `CallupTime` — timestamp when the truck physically entered the terminal.
- `StageStartTime` — timestamp when the activity became ready for execution (enabled time).
- `StageId` — stage identifier (e.g. OCR, Gate-In, yard, Exit).
- `TruckExchangeArea` — visited handling point in the yard for a specific container.
- `TranCtrNbr` — container number associated with the transaction.
- `UnitInboundCarrierId` — inbound carrier identifier (truck plate, vessel voyage number or train ID).

- `UnitOutboundCarrierId` — outbound carrier identifier (truck plate, vessel voyage number or train ID).

A container is considered “picked up” or “delivered” by a truck if the corresponding inbound or outbound carrier identifier resolves to a truck. The report system uses detailed handover-lane codes for the exchange area; these must be mapped to the higher-level operational zones used by the scheduling system. Specifically, all exchange areas beginning with “HP” (*Hochpalette*) are mapped to “LL” (*Leerlager*) to align with the schedule system’s terminology. The mapping is implemented in `002_Prep_area_mapping_report.py`.

CONTAINER MASTER DATA

Container-level attributes are extracted from the container master data and linked to events via the matched `container_id`:

- Container size — TEU equivalent (20’, 40’, 45’).
- Full/empty status — binary indicator (`Leer/Voll`).
- Reefer status — binary indicator for refrigerated containers (derived from the ISO code).
- Hazardous-material status — binary indicator based on the IMCO code.
- Container type — standard, open-top, platform, bulk, etc.
- Dwell time — hours the container has already spent in the terminal (for receive operations).
- Container complexity score — composite metric combining size, full status, reefer, hazmat and dwell time to quantify handling difficulty.

The complexity score is derived during the container-master-data preprocessing stage and combines container type, loading status, hazardous-goods indicators, reefer requirements and other handling-related properties into a single numerical measure. Higher values indicate visits expected to require more complex handling.

YARD STATE DATA

Yard state data provides time-series information on storage-block capacities and utilisation rates.

Each record contains `ZEITPUNKT` (observation timestamp), `AREA` (storage area identifier), `KAP_TEU_THEORETISC`

(theoretical TEU capacity), `BEL_TEU` (current TEU occupancy) and `AUSLASTUNG_%` (utilisation percentage). Because observations are irregularly spaced, the data is resampled to a uniform one-minute frequency using linear interpolation on the numeric fields (script `004_Prep_create_interpolate`).

TRUCK DEMAND DATA

Truck demand information is derived from the scheduling system by aggregating truck arrivals over rolling time windows. For each exchange area and one-minute time interval, the feature `TRUCK_DEMAND_LAST_1H` is calculated as the number of trucks that became ready for processing in that area during the preceding sixty minutes. The demand pipeline (script `005_Prep_create_truck_dem`)

1. Resamples the schedule data to one-minute frequency based on `ALP_ZEITPUNKT_BEREITMELDUNG`.
2. Applies a sixty-minute rolling window to count truck arrivals per area.
3. Produces a separate demand series for each operational area (Gate, To6–T27, VC, LL, MT).
4. Aggregates the individual RMG blocks (To6–T27) into a single yard-section demand metric.

The resulting demand features capture workload pressure at the time of case arrival, which is hypothesised in Section 4.1.2 to influence waiting times and resource-allocation decisions.

3.2.2 DATA CLEANING AND INTEGRATION

CROSS-SYSTEM IDENTIFIER MATCHING

The scheduling and report systems use different identifiers for truck visits. To create a unified event log they are linked through composite keys. Truck visits are matched by `(TruckLicenseNbr, CallupTime)` (`LKW_KENNZEICHEN, FP_ZEITPUNKT_ERSTELLUNG`) using a left join that preserves all schedule records and augments them with the report-system `TruckVisitGkey` when available. Container numbers are then matched via `(TruckVisitGkey, TruckExchangeArea) ≡ (TruckVisitGkey, ANLAU`

For cases where the primary matching fails, a fallback strategy is applied to single-exchange-area visits: if a truck visit contains only one yard activity and no container identifier was matched, the container ID is inferred from any other transaction associated with the same `TruckVisitGkey`. The integration logic is implemented in `01_match_container_ids_to_schedule.py`.

INVALID PROCESS INSTANCES

The cleaning pipeline removes complete cases rather than individual records: whenever a truck visit violates one of the defined quality criteria the entire process instance is excluded, so that only internally consistent process executions are retained. The primary cleaning procedures are implemented in `00_its_fahrplan_raw_to_fahrplan_cleaned_0304.py`.

MISSING LIFECYCLE INFORMATION. Cases are removed if one or more critical lifecycle timestamps are missing: `FP_ZEITPUNKT_ERSTELLUNG`, `ALP_ZEITPUNKT_BEREITMELDUNG` or `ALP_ZEITPUNKT_ER`. Such records cannot be transformed into valid lifecycle events. The quality report identified 1,267 affected cases ($\approx 1.34\%$ of all truck visits).

TEMPORAL INCONSISTENCIES. Cases are removed whenever fundamental temporal constraints are violated. First, activities with negative execution durations (`ALP_ZEITPUNKT_ERFUELLUNG < ALP_ZEITPUNKT_BEREITMELDUNG`) indicate timestamp errors and are excluded. Second, overlapping activities within a single visit ($\text{start}_{i+1} < \text{complete}_i$ after sorting) contradict the operational logic (a truck can only be at one handling location at a time). The cleaning procedure identified three cases with negative durations and 43 with overlapping activities.

EXTREME RUNTIME OUTLIERS. Two further runtime filters are applied. The first removes cases with excessive gate-out delay ($t_{\text{GateOut}} - t_{\text{LastCompletion}} > 10\text{h}$); such records commonly indicate system failure. The second removes visits whose total turnaround time exceeds twelve hours ($t_{\text{GateOut}} - t_{\text{GateIn}} > 12\text{h}$), which is operationally implausible. The filters identified 22 extreme gate-out cases and 22 twelve-hour outliers.

PROCESS COMPLETENESS FILTERS

CONTAINER IDENTIFICATION COVERAGE. Overall, 111,646 of 113,136 operational records could be assigned a valid `TruckVisitGkey` (98.68%), and 110,866 records could be assigned a valid container identifier (97.99%). Despite the fallback procedure, 2,270 records belonging to 2,034 truck visits remained without a valid container assignment. Since container-related attributes are central to the later context enrichment and simulation-discovery stages, these process instances are removed entirely (a complete-case filter). This removes 2,034 truck visits and 3,397 associated records, leaving 91,335 complete truck visits and 109,739 operational records.

UNKNOWN ACTIVITY TYPES. During activity construction, cases containing activities classified as unknown are removed. Such events cannot be reliably mapped to operational process

steps and would introduce ambiguity into both process discovery and simulation discovery.

PROCESS SCOPE RESTRICTION. The event log is restricted to truck visits containing exactly one interaction with an automated RMG block. Visits may still contain additional activities in other yard areas (empty-container yard or VC yard), but only a single RMG block interaction is permitted. This restriction ensures that RMG-specific workload, utilisation and demand measures can be assigned unambiguously at case level. It also forms the basis for the block-focused what-if experiments in Section 4.7.

EVENT-LEVEL CLEANUP. Immediately before exporting the event log, remaining events with an `_unknown` suffix are removed. Unlike the previous filters, this is an event-level cleanup rather than a case-level exclusion; its purpose is to eliminate residual events that cannot be interpreted reliably within the final process model.

3.2.3 CASE DEFINITION

A *case* in the event log corresponds to a single truck visit, uniquely identified by `FAHRPLAN_UID`. Each case represents the complete sequence of activities executed by one truck between gate entry and gate exit.

The data-aware simulation framework used in this thesis [12] accepts only case-level contextual attributes. To attach block-specific context features (`target_utilization`, `target_demand`, `target_rank`) consistently, the design decision was made to include only process instances that interact with exactly one RMG block. Visits involving multiple RMG blocks are excluded because block-specific congestion measures could not be assigned unambiguously. Visits to other yard areas (VC or LL) that also involve exactly one RMG block are retained. The filtering logic is implemented in `filter_one_rmg_only.py`.

3.2.4 ACTIVITY CONSTRUCTION

Activities are defined by combining an *operation type* (receive, delivery or mixed) with a *resource type* (an RMG block, LL or HO2). Table 3.1 lists the resulting labels.

The 22 individual RMG blocks (T06–T27) are retained as distinct resources to preserve block-specific utilisation patterns.

Table 3.1: Activity labels used in the final event log.

Activity	Definition
Gate In	Truck registration at entry gate
RMG_receive	RMG unloads container from truck (import)
RMG_delivery	RMG loads container onto truck (export)
RMG_mixed	Both receive and delivery at same block
LL_receive	Empty-container depot: unload
LL_delivery	Empty-container depot: load
LL_mixed	Empty-container depot: unload and load
H02_receive	Manual area: unload
H02_delivery	Manual area: load
H02_mixed	Manual area: unload and load
Gate Out	Truck departure

3.2.5 ACTIVITY TIMESTAMPS AND TEMPORAL SEMANTICS

The CTB operational systems provide timestamps from which two observed lifecycle states are constructed for each activity:

- Start time (`start:timestamp`) — the moment execution began, mapped from the scheduling or report system.
- Complete time (`time:timestamp`) — the moment execution finished.

The *service time* (or execution time) is the directly identifiable temporal quantity:

$$ET_i = t_{\text{complete},i} - t_{\text{start},i}. \quad (3.1)$$

An explicit `enabled:timestamp` is not reconstructed in the input event log. Consistent with ProSiT [12] and earlier simulation-discovery work [14, 6], activity enablement is obtained from process-model alignment during the simulation-parameter discovery phase (Section 4.4). ProSiT uses an A*-based optimal alignment as defined in Section 2.3.1 to relate each observed event to a transition and determine when that transition becomes enabled in the discovered Petri net, then computes a model-derived pre-service delay relative to both the enablement time and the resource’s availability schedule.

TIMESTAMP MAPPING BY ACTIVITY TYPE

- Gate In. $\text{start} = \text{FP_ZEITPUNKT_ERSTELLUNG}$, $\text{complete} \approx \text{start}$. The OCR gate timestamp (`StageStartTime`) from the report system is retained as the auxiliary column `ocr_timestamp` for diagnostic purposes but is not mapped to an enablement concept.
- Yard activities. $\text{start} = \text{ALP_ZEITPUNKT_BEREITMELDUNG}$, $\text{complete} = \text{ALP_ZEITPUNKT_ERSTELLUNG}$.
- Gate Out. $\text{start} = \text{complete} = \text{FP_ZEITPUNKT_GATEOUT}$.

MODEL-DERIVED PRE-SERVICE DELAY

Because activity enablement is derived from the process model, the resulting quantity

$$D_{\text{pre-service},i} = t_{\text{start},i} - t_{\text{enabled,model},i} \quad (3.2)$$

is a *model-derived pre-service delay*. In ProSiT terminology this is called “waiting time,” but in the CTB application it must not be interpreted as pure queueing or resource-induced waiting. Physical truck movement between consecutive terminal locations is not represented as a separate activity in the discovered process model. Therefore the model-derived interval may include:

- physical movement between operational locations,
- internal traffic and congestion effects,
- resource availability and queue waiting,
- other delays not explicitly represented in the control-flow model.

This is a domain-specific limitation when transferring business-process simulation semantics to physical terminal operations and constitutes an important methodological boundary of this thesis (discussed further in Section 5.10).

METHODOLOGICAL NOTE: THE ABANDONED TRANSITION-BASELINE APPROACH

An earlier version of the event-log pipeline estimated lower-tail route-specific transition durations from the scheduling data (5th percentile of the empirical distribution per transition pair, `transition_baseline.csv`) and used these to reconstruct an explicit `enabled_timestamp` in the input event log. Closer inspection revealed two issues:

1. The statistical route baselines are not event-specific observations of physical arrival; they are aggregate quantiles that vary substantially with the chosen percentile (Section 5.7).
2. In some cases the estimated readiness timestamp occurs after the observed start timestamp, producing negative waiting times that had to be clipped to zero.

Furthermore, ProSiT does not require an externally constructed enabled timestamp — it derives enablement internally from the aligned process model [12]. The final methodology therefore retains only the observed start and completion timestamps and delegates enablement reconstruction to the process-model alignment, improving consistency between data semantics and simulation semantics.

The transition-baseline data is preserved in the repository as diagnostic material and may be useful for future work that introduces an explicit spatial-transition model.

3.2.6 CONTEXT ENRICHMENT

To enable data-aware simulation, the event log is enriched with contextual attributes describing the operational state at the time of process execution. These attributes are organised into five categories.

INFORMATION AVAILABLE AT TRUCK ENTRY

The scheduling record contains the complete planned visit manifest before the truck enters the terminal. Consequently, the number of planned stops, the numbers of delivery and receive transactions, the planned target area and the resulting final visit-complexity score are known at Gate In. The pipeline reproduces this entry-time information by aggregating all rows of the same FAHRPLAN_UID; it does not infer these attributes from subsequently realised durations or completion outcomes. In particular, `visit_complexity` is a deterministic summary of the planned stop/container manifest (visit size, container-handling complexity and mixed-flow complexity), not an ex-post performance measure. Using these variables as case attributes therefore does not leak the simulated future provided that the same planned manifest is supplied for a prospective run.

The implementation detail that the target block is reconstructed from the planned stop's area-specific activity label can otherwise look like look-ahead: after genericising T06--T27 activities, the reconstructed block is copied into `primary_target_area`. Semantically, however, the

source is the planned `ANLAUFUNKT_HALTESTELLE` attached to the visit order and available at entry, not the later activity-completion timestamp. This distinction defines the deployment contract for all entry-known case attributes.

CONTAINER FEATURES

Container-related context is aggregated to case level so that visit properties can serve as case-level attributes in ProSiT (script `03_aggregate_container_details_case_level.py`):

- **n_containers** — number of unique containers handled during the visit.
- **has_hazardous** — at least one IMDG container involved.
- **has_reefer** — at least one reefer container involved.
- **full_ratio** — share of loaded containers.
- **container_complexity_mean** — mean container complexity score.
- **container_complexity_sum** — cumulative complexity score.

Binary indicators are aggregated using logical OR, numerical measures through summary statistics.

YARD FEATURES

Yard state features capture the storage capacity and utilisation status of different terminal areas at truck arrival (Gate In event). These features are matched temporally using `pd.merge_asof` with a nearest strategy:

- `gate_utilization` — overall terminal-yard utilisation.
- `rmg_utilization` — aggregate utilisation across all RMG blocks.
- `vc_utilization` — VC-area utilisation.
- `mt_utilization` — empty-container storage utilisation.
- `target_utilization` — utilisation of the specific block assigned to the truck.

These features are added in `06_add_utils_to_eventlog.py`.

DEMAND FEATURES

Demand features quantify operational load pressure at different terminal areas over a rolling 60-minute window (scripts `05_add_demand_features_to_eventlog.py` and `08_add_all_target_features`).

- `gate_demand` — trucks processed in the last hour across the terminal.
- `rmg_demand` — trucks assigned to RMG blocks in the last hour.
- `vc_demand` — demand for VC operations.
- `mt_demand` — demand for empty-container handling.
- `target_demand` — demand for the assigned RMG block.

TARGET BLOCK FEATURES

For single-block truck visits the assigned target RMG block is a critical attribute. Beyond the target-specific utilisation and demand, the log carries:

- `target_rank` — ordinal ranking of RMG blocks by real-time utilisation at case arrival.
- `target_rank_group` — categorised rank (`top_3`, `mid`, `low`).
- `primary_target_area / target_area` — the block identifier assigned to the case.

These features are reconstructed from the planned, area-specific stop labels before the RMG activities are genericised and are then carried as case attributes (scripts `07_convert_to_RMG_filter_LL+H02_only.py` and `08_add_all_target_features.py`). Their information time is Gate In as specified in Section 3.2.6; the later `org:resource` assignment is not used as an execution outcome to be predicted.

TEMPORAL FEATURES REQUIRED BY PROSiT

ProSiT 1.0.3 does not derive temporal features (hour of day, day of week) directly from event timestamps during simulation-parameter discovery. Time-dependent case attributes must be provided explicitly at case level. Two features are therefore added at the case level:

- `@@hour` — hour of day (0–23) extracted from the earliest `start:timestamp` of the case.

- @@weekday — day of week (0=Mon, 6=Sun).

The @@ prefix indicates that these are case-level attributes used by ProSiT. The temporal-feature extraction is performed in `09_eventlog_to_xes.py`. This explicit encoding of the temporal signal is not an option in ProSiT but a requirement, and it is one of the observed side effects of the ProSiT feature-discovery implementation that are consolidated in Section 4.9.

3.2.7 FINAL EVENT LOG STRUCTURE

The final event log combines process-execution data, lifecycle information and operational context within a single simulation-ready representation. Table 3.2 lists the core process attributes; Tables 3.3–3.6 list the case-level context attributes attached to every event.

Table 3.2: Core process attributes.

Attribute	Description
<code>case:concept:name</code>	Unique truck-visit identifier (FAHRPLAN_UID).
<code>concept:name</code>	Activity executed during the visit.
<code>start:timestamp</code>	Observed/reconstructed activity start timestamp.
<code>time:timestamp</code>	Observed/reconstructed activity completion timestamp.
<code>org:resource</code>	Operational location of the activity.

Table 3.3: Case-level container and process-shape attributes.

Attribute	Description
<code>process_flow_type</code>	Visit type (delivery, receive, mixed).
<code>n_containers</code>	Number of containers in the visit.
<code>n_stops</code>	Number of planned operational stops, known at Gate In.
<code>n_deliveries</code>	Number of planned delivery transactions, known at Gate In.
<code>n_receives</code>	Number of planned receive transactions, known at Gate In.
<code>has_hazardous</code>	IMDG indicator.
<code>has_reefer</code>	Reefer indicator.
<code>full_ratio</code>	Share of loaded containers.
<code>visit_complexity</code>	Final composite score of the complete planned visit manifest, known at Gate In.

Table 3.4: Demand and utilisation features at case arrival.

Attribute	Description
gate_demand	Current gate demand.
rmg_demand	Current demand in the RMG yard.
vc_demand	Current demand in the VC area.
mt_demand	Current demand in the manual yard.
gate_utilization	Current gate utilisation.
rmg_utilization	Current RMG-yard utilisation.
vc_utilization	Current VC-area utilisation.
mt_utilization	Current manual-yard utilisation.

Table 3.5: Target-area context attributes.

Attribute	Description
target_utilization	Utilisation of the primary target area at truck arrival.
target_demand	Demand for the primary target area.
target_rank	Utilisation rank among all RMG blocks.
target_rank_group	Categorised rank (top_3, mid, low).
primary_target_area / target_area	Planned target area known at Gate In.

All contextual attributes are attached at case level and therefore remain constant throughout the case. This design is consistent with the current ProSiT feature-discovery approach, which primarily utilises case-level context information rather than event-level dynamic state [12].

3.2.8 EVENT LOG CHARACTERISTICS

After the full preprocessing pipeline the final event log contains 89,460 truck visits represented by 275,408 events. The activity distribution is shown in Table 3.7. Table 3.8 summarises the overall event log.

Every case contains a Gate In event, one or more operational yard activities and a final Gate Out event. Operational activities may occur at an RMG block, the LL area, HO2 or the VC area, depending on the operational requirements of the visit.

The generated event log is exported as the full CSV `s6_eventlog_target_rank_features.csv`. The frozen temporal partitions are exported as `s6_train.csv`, `s6_test.csv` and the corresponding `xes_files/s6_train.xes` and `xes_files/s6_test.xes`. The CSV represen-

Table 3.6: Temporal context attributes required by ProSiT.

Attribute	Description
@hour	Hour of day derived from the case-start timestamp.
@weekday	Day of week derived from the case-start timestamp.

Table 3.7: Activity distribution in the final event log.

Activity	Frequency
Gate In	89,460
Gate Out	89,460
RMG_receive	39,965
RMG_delivery	22,504
LL_delivery	13,230
HO ₂ _delivery	7,224
HO ₂ _receive	5,464
RMG_mixed	4,939
LL_receive	2,073
HO ₂ _mixed	1,025
LL_mixed	64

tation serves as the primary pipeline format; XES enables compatibility with pm4py, ProM and ProSiT for process discovery and conformance analysis.

The chronological character of the log is preserved end to end: every case carries the same monotone timeline from Gate In to Gate Out. This property is exploited in Chapter 4 to define a temporal train/test split at case level (see Section 4.2).

Table 3.8: Final event log summary.

Statistic	Value
Cases	89,460
Events	275,408
Distinct activities	11
Observation start	2026-03-02 05:27
Observation end	2026-04-30 11:54
Distinct RMG resources	22 (T06–T27)
Additional pools	LL, HO2, Gate In, Gate Out

4

Experiments and Evaluation Framework

This chapter defines what is being evaluated, how it is evaluated, and what evidence the reader should expect in Chapter 5. It is structured so that every experimental choice can be traced to a concrete question and a concrete artefact in the repository.

Section 4.1 restates the research questions and derives falsifiable hypotheses that Chapter 5 will accept or reject. Section 4.2 defines the held-out train/test split that separates discovery from evaluation. Section 4.3 describes the control-flow discovery procedure and its conformance protocol. Section 4.4 defines the three ProSiT configurations that will be compared: distribution-based (no-rules), data-aware with decision trees (rules) and data-aware with additional queue-length features (rules+workload). Section 4.5 defines the multi-seed Monte-Carlo replication protocol that produces the confidence intervals used throughout Chapter 5. Section 4.6 defines the fidelity metrics. Section 4.7 defines the two operational what-if scenarios. Section 4.8 defines the transition-baseline sensitivity study. Section 4.9 documents the deviations from a clean ProSiT invocation that were required to complete the experiments on the CTB event log.

Figure 4.1 gives an overview of the full experimental pipeline. Every stage in the diagram is realised by a script in the code repository; the section-level cross-references in the caption identify the concrete implementation.

Raw
Operational
Data



Eventlog
Engineering



Process
Discovery



Simulation
Discovery

4.1 RESEARCH QUESTIONS, HYPOTHESES AND FRAMING

Chapter 1 formulated the overarching research questions. To make them empirically tractable, they are here refined into testable claims. The claims deliberately admit rejection: an outcome that contradicts a hypothesis is a valid contribution and is reported as such in Chapter 5.

4.1.1 REFINED RESEARCH QUESTIONS

RQ₁ (Fidelity). To what extent does a ProSiT simulator discovered from historical CTB truck data reproduce case-level turnaround, activity durations, waiting times and arrivals on a held-out temporal test window?

RQ₂ (Structural quality). Does the discovered Petri net generalise to the held-out period, or is fitness only preserved on the training period?

RQ₃ (Modelling choices). How sensitive is simulation fidelity to three plausible ProSiT configurations (*no-rules*, *rules*, *rules+workload*)?

RQ₄ (Scenario support). Does the discovered simulator produce operationally plausible KPI shifts under two transparent interventions: closing a single RMG block and increasing truck demand by 20 %?

4.1.2 HYPOTHESES

H₁ (Time dependence of arrivals). A time-aware arrival model (decision tree over hour, week-day, weekend indicator) reduces inter-arrival prediction error against a pooled-rate baseline on cross-validated hold-out slices from the training log.

H₂ (Data-aware fidelity gain). Enabling ProSiT decision-tree rules (*rules*) produces lower distributional distance to the held-out test log than the distribution-only ProSiT (*no-rules*) for at least one of case-turnaround EMD, waiting-time EMD or service-time EMD.

H₃ (Workload-feature gain). Adding queue-length and workload-history features to the ProSiT resource-selection and waiting-time models (*rules+workload*) further reduces distributional distance versus *rules* on the held-out test log.

H4 (Scenario response). Closing T22 eliminates assignments to that resource and increases RMG pre-service delay, while a 20 % increase in working-time arrival intensity measurably increases the observed truck-arrival rate. Downstream turnaround effects are estimated rather than assumed.

Every hypothesis is intended to be falsifiable. In particular H2 and H3 are treated as claims of *practical* benefit: the point estimate must be lower and the 95 % confidence interval derived in Section 4.5 must not overlap the reference configuration entirely for the improvement to be reported as a fidelity gain.

4.2 HELD-OUT TRAIN/TEST SPLIT

4.2.1 RATIONALE

A held-out validation set is required to test H1–H3. In simulation discovery it is not sufficient to report the fit of parameters on the data used to discover them: a well-fit ProSiT bundle can memorise the training distribution while over-fitting the operational dynamics. The ProSiT reference implementation [12] describes an in-memory split in its *Advanced Usage* example, but does not persist the split, does not commit to a case-level boundary and does not report validation metrics. This thesis therefore uses a persistent, case-level, temporally ordered split that any downstream script can consume unchanged.

4.2.2 SPLIT DEFINITION

Let $\mathcal{L} = (c_1, c_2, \dots, c_N)$ be the set of $N = 89,460$ cases in the final event log. For each case c_i let $\tau(c_i)$ denote its earliest event timestamp:

$$\tau(c_i) = \min\{\text{start}(e), \text{time}(e) \mid e \in c_i\}. \quad (4.1)$$

Sort the cases by τ and a deterministic case-identifier tiebreaker. The first 80 % form the training set and the remaining 20 % form the test set. The cutoff τ^* is recorded as the earliest arrival in the test set. Cases whose earliest event timestamp is missing are dropped from both.

On the CTB event log the cutoff is $\tau^* = 2026-04-20\ 18:17:00$. The split produces 71,568 training cases (220,048 events) and 17,892 testing cases (55,360 events); no events are dropped. The

split is realised by `validation/01_train_test_split.py` and is documented in `validation/results/sp` which also records the seed, the cutoff and the per-activity coverage in both partitions. The manifest is loaded by every downstream simulation script so that n_{traces} and t_{start} match the test window exactly.

4.2.3 CORRECTION RELATIVE TO EARLIER DRAFTS

Earlier versions of this thesis referenced a training set of 24,000 cases and a test set of 6,000 cases. Those numbers were an artefact of a legacy random-subsample step in `baseline/export_empirical_duration` that only fired when the discovery scripts were pointed at the full event log. All discovery scripts are now routed through `baseline/_eventlog_source.py`, which prefers the persisted `s6_train.csv` and skips the legacy subsample when a train log is detected. The correct partitioning is therefore 71,568/17,892 as recorded in the split manifest.

4.3 PROCESS DISCOVERY PROTOCOL

4.3.1 ALGORITHM

The Petri net that ProSiT uses as the authoritative control-flow skeleton is constructed from the observed training variants as a sequential prefix trie, following the definition in Chapter 2. Every accepted path has the form `Gate In` $\rightarrow$ one or more yard activities $\rightarrow$ `Gate Out`. Shared prefixes are represented once, and a separate transition encodes each observed continuation [23, 22]. In the Petri-net encoding, each prefix state becomes a place and each observed continuation becomes a labelled transition to the successor prefix state. All complete paths terminate in the common sink after `Gate Out`. The resulting net is acyclic and has one token per truck case.

4.3.2 WHY THE INDUCTIVE-MINER NET IS DIAGNOSTIC ONLY

The Inductive Miner is not rejected as a discovery method: its sound, block-structured models and abstraction from infrequent variants are valuable when behavioural generalisation is the objective [21]. It is retained here as a diagnostic process tree and BPMN visualisation. It is not passed to the final simulator because its discovered semantics conflict with the CTB case contract. In particular, the diagnostic tree in Figure 2.1a contains parallel (and) cuts, including a cut

that places Gate Out in parallel with the yard subtree. Executing this model could therefore permit Gate Out before all yard work is complete or enable multiple yard activities within one truck visit. These behaviours may be sound in the formal workflow-net sense, but they are not valid truck-process semantics in this application.

The trie makes this domain constraint explicit: exactly one Gate In is followed by one or more sequential yard activities and exactly one final Gate Out. This does *not* disable resource multitasking between trucks. Different Petri-net instances still compete for and can use the same resources concurrently according to the discovered capacities; only the activities within a single truck case remain sequential. In the final CTB baseline these cross-case RMG capacities are subject to the domain cap of three described below.

This choice is deliberately conservative. A raw observed-variant trie does not generalise to every unseen activity combination and is less compact than an Inductive-Miner model. The held-out conformance protocol below therefore tests whether this loss of permissiveness is material. The authoritative trie is implemented by `_eventlog_contract.py` and injected by `baseline/07_run_prosit_discovery.py`. It contains 76 places and 144 transitions for the 70 variants observed in the frozen training partition.

4.3.3 CONFORMANCE PROTOCOL

Control-flow conformance metrics are computed against *both* the training and the held-out testing partition:

- Token-based replay fitness, i.e. the fraction of traces that the Petri net can replay without token deficit.
- Token-based replay precision, the share of behaviour allowed by the model that is also observed in the log.
- Generalisation via `pm4py.generalization_tbr`.
- Simplicity via `pm4py.simplicity_petri_net`.

They are supplemented by an explicit case-language audit: exactly one initial Gate In, at least one yard activity, exactly one final Gate Out, no within-case activity overlap and monotonically non-decreasing completion timestamps. This separates two questions that aggregate fitness alone cannot answer: whether the net replays held-out variants and whether the generated

traces obey the physical truck-visit order. The full protocol is implemented in `baseline/07_run_prosit_disc` and `_eventlog_contract.py`.

4.4 SIMULATION PARAMETER DISCOVERY CONFIGURATIONS

ProSiT is used unchanged as the discovery and simulation engine [12]. All simulation parameters are learned by `SimulatorParameters.discover_from_eventlog` from the training partition only. Three configurations are evaluated. They are chosen so that H₂ (data-aware fidelity gain) and H₃ (workload-feature gain) each correspond to exactly one comparison.

4.4.1 CONFIGURATION *NO-RULES*

The *no-rules* configuration uses `max_depth_tree= 0`, which disables the ProSiT decision-tree mechanism entirely. Every parameter reduces to a single unconditional distribution or a marginal frequency: one arrival-rate distribution, one execution-time distribution per activity, one waiting-time distribution per resource, one categorical distribution over routing choices at each Petri-net decision point. This configuration is the distribution-based baseline against which H₂ is tested.

4.4.2 CONFIGURATION *RULES*

The *rules* configuration uses the ProSiT default (`max_depth_tree= 3`, `min_samples_leaf_cv= [50, 100, 200]`, `random_state= 42`). Every timing, routing and resource-selection model is fitted as a decision tree whose leaves carry the local distributions. Cross-validation over the leaf-size grid selects the actual leaf size per model, and models that fail to beat the marginal distribution on held-out CRPS collapse to the pooled baseline (this is a ProSiT internal safeguard against overfitting to noise [12]). Case-level attributes made available to the trees are the container, demand, utilisation, target-area and temporal features listed in Section 3.2.7.

4.4.3 CONFIGURATION *RULES+WORKLOAD*

The *rules+workload* configuration is identical to *rules* but with `use_workload_features=True`. As documented in the ProSiT README this enables two extra features on the waiting-time

and resource-selection models: current workload (number of concurrent tasks) and queue length (number of tasks scheduled but not yet started) at the time each event is enabled in the process model. The intention behind including this configuration is to let ProSiT see the queueing state that its distribution family alone would otherwise ignore. This configuration is compared against *rules* to test H₃.

4.4.4 DISCOVERY HYPERPARAMETERS

Table 4.1 lists the ProSiT hyperparameters that differ across configurations. All other arguments are the ProSiT defaults except `attribute_mode`, which is set to 'empirical' for reasons detailed in Section 4.9.

Table 4.1: ProSiT discovery hyperparameters by configuration.

Hyperparameter	no-rules	rules	rules+workload
<code>max_depth_tree</code>	0	3	3
<code>min_samples_leaf_cv</code>	–	[50, 100, 200]	[50, 100, 200]
<code>use_workload_features</code>	False	False	True
<code>attribute_mode</code>	empirical	empirical	empirical
<code>random_state</code>	42	42	42

The full pipeline is executed by `baseline/07_run_prosit_discovery.py`. The pickled `SimulatorParameters` objects (`prosit_params.pkl`) and their JSON representations are stored in the paramset folder `baseline/discovery_params/params_<utc>_train80/prosit_discovery_no_rules|_workload|/`.

4.4.5 AUTHORITATIVE PARAMETER BUNDLE AND SERIALIZATION CON-TRACT

In this thesis, *authoritative parameter bundle* is an operational reproducibility term: it denotes the exact executable `SimulatorParameters` state loaded by the validation and scenario scripts. If a derived table, JSON summary and executable state disagree, the serialized object that actually generated the reported simulations is the source of truth. Two provenance layers are distinguished. The frozen, training-only calibrated *rules+workload* `prosit_params.pkl` is authoritative for the freely discovered and calibrated source model. The final CTB baseline

is a deep-copied effective bundle in which the RMG concurrency values are capped at the domain maximum of three; it is stored as `params_baseline_rmg_max_concurrency_3.pkl` in the final scenario result directory and is authoritative for the final validation and scenario results. Thus the discovered overlap capacities remain auditable without being mistaken for physically feasible capacities. Both objects contain the Petri net and markings, routing and resource-selection rules, resource mappings and capacities, calendars, distribution objects, calibration metadata and the empirical sample arrays used for stochastic draws.

The ProSiT JSON export is retained as a portable, human-inspectable companion, but it is not an execution-equivalent serialization of the calibrated object. In particular, the JSON rule nodes preserve fitted distribution names, parameters and representative values but do not serialize all empirical sampled arrays held in the Python object. This matters because the CTB calibration modifies those arrays and ProSiT samples from them at runtime. Consequently, reconstructing the final simulator from JSON alone can change its stochastic behaviour. The pickle is therefore authoritative because the JSON does not fully represent all empirical sample arrays; this statement concerns the actual ProSiT serialization schema used here, not an intrinsic superiority of the pickle format.

The companion files `prosit_run_summary.json`, `control_flow_contract.json` and `ctb_calibration.json` document provenance and intended changes. Validation and scenario scripts record SHA-256 hashes of their source pickles, and scenario bundles are produced by deep copy rather than by mutating the frozen baseline. Pickle is Python- and class-version-specific and must never be loaded from an untrusted source; reproducibility therefore also requires the recorded software environment. These limitations are the reason for retaining the JSON companions even though they are not the authoritative runtime state.

4.5 MONTE-CARLO REPLICATION PROTOCOL

Every ProSiT simulation includes stochastic draws from the fitted distributions. Reporting a single Monte-Carlo draw is therefore insufficient to conclude that one configuration is more faithful than another: the observed difference may be within the sampling variance of the simulator itself. This thesis therefore reports every fidelity metric with a Monte-Carlo confidence interval.

4.5.1 REPLICATION DESIGN

For each ProSiT configuration $\text{cfg} \in \{\text{no-rules}, \text{rules}, \text{rules+workload}\}$ the following procedure is executed:

1. Load the corresponding `SimulatorParameters` bundle from disk.
2. Instantiate a fresh `SimulatorEngine`.
3. For $i = 0, \dots, N_{\text{seeds}} - 1$:
 - (a) Seed Python's `random` and `numpy.random` with `base + i`.
 - (b) Simulate $n_{\text{traces}} = 17,892$ cases starting at $t_{\text{start}} = 2026-04-20\ 18:17:00$, i.e. the test-window cutoff read from `split_manifest.json`.
 - (c) Compute the fidelity metrics of Section 4.6 against the held-out test log.
4. Aggregate the N_{seeds} replications into a Student-t 95 % confidence interval per metric:
 $\bar{x} \pm t_{0.975, N-1} s / \sqrt{N}$.

4.5.2 IMPLEMENTATION

The protocol is implemented in `validation/06_multi_seed_ci.py`. $N_{\text{seeds}} = 10$ is used for every configuration, following the sample-size heuristic that a Student-t interval on four KPIs is meaningful at $N = 10$ while remaining computationally affordable at ~ 50 seconds per simulation. The base seed is 42.

The choice to seed Python's `random` and `numpy.random` before each `engine.apply()` rather than the ProSiT engine itself is intentional: ProSiT 1.0.3 does not expose a per-call seed on its simulation engine. Seeding both random-number generators before the call yields reproducible replications and captures the same stochastic variability that a user would observe in production.

4.6 FIDELITY METRICS

The comparison of a simulated event log against the real held-out log uses four distributional distance metrics. Each metric is computed once per Monte-Carlo replication so that a confi-

dence interval can be attached to the point estimate.

4.6.1 WASSERSTEIN DISTANCE (EMD)

For each yard activity a the service-time EMD is

$$\text{EMD}_a^{\text{svc}} = W_1(F_a^{\text{sim,svc}}, F_a^{\text{real,svc}}), \quad (4.2)$$

where $F_a^{\text{sim,svc}}$ and $F_a^{\text{real,svc}}$ are the empirical CDFs of time — start for activity a in the simulated and real log, respectively. Gate In and Gate Out are excluded from the service-fidelity aggregation because both have zero-duration timestamp constructions and would mechanically improve the score without representing terminal handling. Let $\mathcal{A}_Y$ be the set of yard activities and n_a^{real} the number of valid held-out observations. This thesis reports all per-activity values and two yard-level aggregates:

$$\overline{\text{EMD}}_Y = \frac{1}{|\mathcal{A}_Y|} \sum_{a \in \mathcal{A}_Y} \text{EMD}_a^{\text{svc}}, \quad (4.3)$$

$$\overline{\text{EMD}}_{Y,w} = \frac{\sum_{a \in \mathcal{A}_Y} n_a^{\text{real}} \text{EMD}_a^{\text{svc}}}{\sum_{a \in \mathcal{A}_Y} n_a^{\text{real}}}. \quad (4.4)$$

The first gives every yard-activity class equal influence; the second describes the error experienced by the empirical mix of yard events. The case-level turnaround EMD is $W_1(F^{\text{sim,ttot}}, F^{\text{real,ttot}})$ where ttot is the total elapsed time per case (first start to last complete). The inter-arrival EMD is $W_1(F^{\text{sim,iat}}, F^{\text{real,iat}})$ where iat is the time between consecutive case-arrival timestamps. All EMDs are reported in minutes. The primary view retains durations up to 24 hours and is accompanied by an uncapped raw view so that an artificial simulation tail cannot be hidden.

4.6.2 KOLMOGOROV–SMIRNOV STATISTIC

For each distribution pair the two-sample KS statistic is computed by `scipy.stats.ks_2samp`. Because the CTB test log contains 55,360 events, sub-sampling is applied so that each side contributes at most 10,000 observations to the test. This keeps the p-value informative and prevents the null from being rejected mechanically by sample size. The KS statistic (magnitude of the maximum vertical CDF gap) is reported alongside the EMD; the p-value is retained for the appendix.

4.6.3 POINT-STATISTIC ERRORS

For each activity and each case-level KPI the mean, median and 90th percentile of the simulated and real distributions are compared as absolute and relative errors. This complements the distributional metrics with operationally interpretable point statistics (e.g. “the simulator under-predicts mean turnaround by X minutes”).

4.6.4 IMPLEMENTATION

The metric definitions are implemented in `validation/02_validate_simulation.py` and reused verbatim by `validation/06_multi_seed_ci.py` to guarantee that single-shot and multi-seed numbers are directly comparable.

4.7 WHAT-IF SCENARIO DESIGN

Two independent interventions are evaluated to test H4. Both are derived by deep copy from the frozen, training-only calibrated *rules+workload* baseline. No held-out observation is used to select or fit a scenario parameter.

4.7.1 SCENARIO A: T22 RESOURCE-POOL REALLOCATION

Scenario A removes T22 from the simulated RMG resource pool. The scenario is constructed by (i) removing T22 from every `params.act_to_resources` entry that references it, after verifying that each affected activity retains at least one eligible resource; and (ii) setting all 168 weekly calendar slots of T22 to unavailable. All other parameters are unchanged.

This is explicitly a resource-pool reallocation experiment, not a complete spatial yard-closure model. The discovered selector may use target-area attributes as statistical predictors, but ProSiT does not impose a hard constraint that binds a container to its actually observed storage block. A truck whose historical target was T22 can therefore be assigned to another eligible block without an explicit container-relocation operation or a location-dependent travel-time adjustment. This boundary is evaluated explicitly in Section 5.10.

4.7.2 SCENARIO B: 20 % HIGHER TRUCK DEMAND

Scenario B changes only the empirical working-time inter-arrival distribution. For demand multiplier $m = 1.20$, every empirical inter-arrival sample x_i is replaced by x_i/m . This preserves the zero mass and the empirical distribution shape while increasing the modelled arrival intensity by exactly 20 %. The gate-admission calendar, process model, duration models, routing, resource mappings, resource calendars and case-attribute schema remain unchanged. The scenario is independent of the T22 resource-pool reallocation.

4.7.3 FINAL DOMAIN-CONSTRAINED RMG CAPACITY

The freely discovered serialized source assigns maximum concurrency four to ten RMG block identifiers and five to twelve, for an aggregate effective upper bound of 100. ProSiT obtains these numbers from overlapping timestamps; they are empirical concurrency proxies rather than equipment counts. They exceed the hard physical upper bound of three cranes and the three lanes in which trucks can wait after announcing readiness at a block. The final effective baseline is therefore derived by replacing every RMG value c_r with $\min(c_r, 3)$. The aggregate upper bound becomes $22 \times 3 = 66$. This does not assert that three cranes normally serve landside trucks: in regular operation two cranes may serve the waterside and at most one may serve the landside, but that allocation depends on ship operations and is not observed in this event log. Three is therefore a defensible physical ceiling, not an estimated landside staffing level. No resource mapping, calendar, selection rule, timing model, arrival model or control-flow parameter is changed. The T22 and demand interventions are then independently re-derived from this capped baseline.

This is a transparent domain constraint rather than a data-driven recalibration: the freely discovered source remains frozen as discovery evidence, whereas the cap-three copy is the authoritative effective CTB baseline. An uncapped run is retained only as a diagnostic sensitivity to show what the overlap-derived parameter would imply. Both sets of replications use identical seeds, trace counts and start time. In addition to direct cap effects, the diagnostic comparison reports the interaction

$$\Delta_{\text{interaction}} = (Y_{\text{scenario}} - Y_{\text{baseline}})_{c \leq 3} - (Y_{\text{scenario}} - Y_{\text{baseline}})_{\text{discovered}}, \quad (4.5)$$

so that changes in scenario response are not confused with a general level shift caused by the

cap.

4.7.4 REPORTING

The unified driver `validation/09_multi_seed_scenarios.py` executes baseline and both interventions with ten matched seeds (42–51), $n_{\text{traces}} = 17,892$ and $t_{\text{start}} = 2026-04-20\ 18:17$. Thus all paired deltas use common random numbers. Mean, median and 90th percentile of turnaround, RMG service and RMG pre-service delay are reported together with the observed arrival KPIs. For the T22 experiment, service and pre-service effects are additionally reported separately for `RMG_receive` and `RMG_delivery`. Every one of the 30 generated logs is subjected to the sequential case contract of Section 4.3; any violation aborts the run. A separate diagnostic invocation with `--no-rmg-cap` reproduces the 30 uncapped overlap-capacity replications. `validation/10_compare_rmg_capacity_sensitivity.py` verifies identical source hashes and settings before computing the paired direct effects and interactions. These uncapped results are diagnostic only and are excluded from the authoritative final result table; the 30 cap-three scenario logs form the final scenario evidence.

4.8 DESCRIPTIVE TRANSITION-DURATION ANALYSIS

As discussed in Section 3.2.5, an earlier pipeline version reconstructed activity enablement from route-specific transition baselines. Although this approach was abandoned in favour of ProSiT’s internal model-derived enablement, the empirical transition durations remain informative as descriptive evidence of the physical separation between terminal activities.

To characterise this physical component, the same baseline table is recomputed with the 5th, 10th, 25th and 50th percentiles as alternative choices. For each transition (a, b) with at least twenty observations the resulting baseline duration is reported together with the shift relative to the 5th percentile and the rank correlation between percentiles. This analysis is implemented in `validation/04_baseline_percentile_sensitivity.py` and reported in Section 5.7. It does not affect ProSiT’s simulation-parameter discovery; its purpose is to illustrate that the model-derived pre-service delay includes a non-trivial physical-movement component that cannot be separated from operational waiting with the available data.

4.9 PROSiT IMPLEMENTATION NOTES AND WORKAROUNDS

Applying ProSiT 1.0.3 [12] to the CTB event log uncovered several implementation issues that would have propagated as silent misbehaviour if not addressed. This section documents them so that the experiments can be reproduced deterministically and so that the downstream fidelity numbers can be interpreted correctly.

4.9.1 DATA-ATTRIBUTE DISCOVERY ON NAN COLUMNS

The default `attribute_mode='distribution'` passes each case attribute through a per-attribute distribution fitter. In the CTB log, `full_ratio` and `is_full` contain NaN for cases that involve empty-container-only visits. The distribution fitter in ProSiT 1.0.3 forwards the raw values to `scipy.stats`, which raises `TypeError: ufunc 'isfinite' not supported on the mixed dtype`. Switching to `attribute_mode='empirical'` bypasses the per-attribute fitter and samples full attribute tuples from the observed joint distribution instead. This is not a fidelity compromise: the discovered simulator still preserves case-level attribute correlations, and empirical sampling from the training distribution is the appropriate choice given the number of categorical attributes present in the CTB log.

4.9.2 SIMULATION IN NO-RULES MODE

In *no-rules* mode ProSiT samples the routing choice via `random.choice(seq)` where `seq` is a `numpy.ndarray`. Python's built-in `random.choice` evaluates `if not seq:`, which raises `ValueError: The truth value of an array with more than one element is ambiguous.` on multi-element numpy arrays. The workaround is a minimal monkey patch that intercepts `random.choice` while the engine is running and forwards `numpy.ndarray` arguments through `numpy.random.randint`. The patch is contained to the simulation call and is restored immediately afterwards. It is applied by both `baseline/07_run_prosit_discovery.py` and `validation/06_multi_seed_ci.py`.

4.9.3 DUPLICATED COLUMNS IN THE SIMULATED LOG

ProSiT 1.0.3 writes case-level attributes into the event-level frame returned by `SimulatorEngine.apply()`. When a case-level attribute has the same name as an event attribute (in the CTB log this hap-

pens for `enabled:timestamp` because ProSiT writes case-level attributes into the event-level frame), the resulting `pandas.DataFrame` carries duplicate column labels. Standard operations such as `pd.to_datetime(df[col])` then return a `DataFrame` slice instead of a `Series` and fail. The workaround is a single `df.loc[:, ~df.columns.duplicated()]` call at the start of every KPI-computation function. It is applied both in the what-if scripts and in the multi-seed CI driver.

4.9.4 ABSENCE OF AUTOMATIC TEMPORAL FEATURES

ProSiT does not derive hour-of-day or day-of-week features from the event-log timestamps during simulation-parameter discovery: temporal features must be provided as explicit case attributes. This is why the event-log engineering pipeline of Section 3.2.6 adds `@@hour` and `@@weekday` at case level. Without them, ProSiT’s arrival-time and duration models cannot condition on the strong day/night pattern that Section 5.3 documents.

4.9.5 NON-DETERMINISTIC SIMULATION ENGINE

`SimulatorEngine.apply()` does not accept an explicit seed. The Monte-Carlo protocol of Section 4.5 therefore seeds Python’s `random` and `numpy.random` before every call to `apply()`. Two consecutive calls with different seeds produce different logs; two consecutive calls with the same seed produce the same log up to floating-point noise. This is the reproducibility guarantee assumed throughout Chapter 5.

4.9.6 CONTRIBUTION BOUNDARY

The ProSiT engine itself was not modified for these workarounds. The CTB pipeline patches its own callers so that ProSiT can be invoked without source-code changes. All five workarounds together constitute the technical adaptation of ProSiT to the CTB case study and are listed here so that a reader can distinguish the contribution of this thesis (case-study specific data engineering, held-out validation methodology, multi-seed replication, sensitivity analysis, what-if scenario protocol) from the contribution of ProSiT itself (automatic simulation-parameter discovery from event logs [12]).

5

Results

This chapter reports the empirical evidence produced by the pipeline defined in Chapter 4. Every number in this chapter is traceable to a script and an artefact in the repository; every claim is qualified by its confidence interval or its inherent uncertainty. The chapter is organised so that the reader encounters the evidence in the same order in which the hypotheses of Section 4.1.2 depend on it.

Section 5.1 reports the descriptive characteristics of the training and testing partitions produced by the split of Section 4.2. Section 5.2 reports the discovered Petri net and its conformance on both partitions (RQ2). Section 5.3 reports the temporal structure of the arrival process (H1). Section 5.4 reports the held-out Monte-Carlo fidelity of the ProSiT configurations (RQ1, H2, H3). Section 5.5 decomposes the fidelity by activity to identify where each configuration wins or loses. Section 5.6 reports the discovered resource capacities and their limits. Section 5.7 provides a descriptive analysis of transition durations. Section 5.8 reports the two what-if scenarios (H4). Section 5.10 discusses the methodological limitations. Section 5.9 consolidates the empirical findings against the hypotheses.

5.1 EVENT LOG CHARACTERISTICS ON THE HELD-OUT SPLIT

The train/test split of Section 4.2 yields a temporally ordered partition of the 89,460 cases. The training log contains the first 71,568 cases (220,048 events; Gate In arrivals from 2026-03-02 05:27 through 2026-04-20 18:15). The testing log starts with the next observed arrival at the frozen cutoff 2026-04-20 18:17 and contains 17,892 cases (55,360 events). No events are dropped for missing timestamps. Table 5.1 lists the per-activity counts in both partitions.

Table 5.1: Activity counts on the train and test partitions.

Activity	Train	Test	Total
Gate In	71,568	17,892	89,460
Gate Out	71,568	17,892	89,460
RMG_receive	32,794	7,171	39,965
RMG_delivery	17,823	4,681	22,504
RMG_mixed	3,986	953	4,939
LL_delivery	9,612	3,618	13,230
HO2_delivery	5,775	1,449	7,224
HO2_receive	4,188	1,276	5,464
HO2_mixed	749	276	1,025
LL_receive	1,932	141	2,073
LL_mixed	53	11	64

No activity is missing from the test partition, but the temporal split does expose genuine distribution shift. For example, LL_delivery rises from 0.134 to 0.202 events per case, whereas RMG_receive falls from 0.458 to 0.401 and LL_receive from 0.027 to 0.008. This shift is retained rather than tuned away: it is part of the held-out generalisation challenge and explains most of the final yard-activity-rate L1 error reported in Table 5.10.

5.2 PROCESS DISCOVERY AND HELD-OUT CONFORMANCE

The authoritative simulation net is the sequential observed-variant trie described in Section 4.3. It contains 76 places and 144 transitions and represents 70 training variants. Unlike the diagnostic inductive process tree, it cannot enable two yard activities within the same truck case concurrently. Cross-case resource multitasking remains enabled. Table 5.2 reports conformance on

the frozen partitions.

Table 5.2: Petri-net conformance on train and test.

Metric	Train	Test
Log fitness	1.0000	0.99985
Fitting traces (%)	100.000	99.944
Average trace fitness	1.0000	0.99990
Token-based precision	1.0000	0.96057
Generalisation	0.57677	0.37327
Simplicity	0.61798	0.61798

The test partition contains 53 variants. Ten cases follow ten variants that were not observed in training, corresponding to only 0.0559 % of the 17,892 test cases. This accounts for the small fitness deviation from one. The lower generalisation score is expected from a strict variant trie: the design intentionally trades permissiveness for an auditable case language of exactly one Gate In, one or more sequential yard activities and exactly one final Gate Out. It does not imply that yard resources cannot work on different trucks simultaneously.

RQ2 (structural quality). The authoritative net replays 99.944 % of held-out traces perfectly and achieves log fitness 0.99985 while enforcing the required within-case sequence. It is therefore a defensible simulation skeleton for this application. Its strict variant representation, and the resulting generalisation score, remain an explicit modelling trade-off.

5.3 TEMPORAL STRUCTURE OF THE ARRIVAL PROCESS

The arrival process is the clearest case for time-conditioning in the CTB event log. Table 5.3 compares a pooled arrival-rate baseline (a single global mean) against a time-aware decision tree (features: hour, weekday, weekend indicator) on 30-minute arrival counts derived from the training partition. Both models are evaluated on a chronological 80/20 split within the training partition so that the numbers reflect the same temporal-hold-out logic used for the simulator evaluation later.

The pooled baseline has an R^2 indistinguishable from zero: a single global arrival rate has no predictive power for 30-minute arrival counts on this dataset. The time-aware tree reduces MAE by 68.8 % and RMSE by 52.8 % and achieves $R^2 = 0.777$. The day/night statistics of

Table 5.3: Arrival-rate benchmark on 30-minute buckets (training log only, chronological 80/20 split).

Model	MAE	RMSE	R ²
Pooled arrival rate	28.98	34.19	-0.001
Time-aware decision tree	9.04	16.15	0.777

Table 5.4 confirm the underlying non-stationarity: mean arrivals per hour rise from 12.7 at night to 110.8 during the day, an $8.7\times$ difference that a pooled model cannot represent.

Table 5.4: Day vs. night arrival statistics on the training log.

Period	Mean inter-arrival (min)	Mean arrivals/hour	Cases
All	1.00	60.2	71,568
Day	0.54	110.8	63,758
Night	4.71	12.7	7,810

H₁ (time dependence of arrivals) is accepted. The time-aware arrival model reduces MAE by 68.8 % and RMSE by 52.8 % relative to a pooled baseline on a chronological within-training hold-out. This is a substantial and interpretable effect. The day/night contrast (Table 5.4) is the operational mechanism that the tree captures.

5.4 HELD-OUT SIMULATION FIDELITY (MULTI-SEED)

The ProSiT configurations of Section 4.4 were first screened against a temporal hold-out. The endpoint comparison is retained in Table 5.5 as evidence for H₂ and H₃. After that comparison, the workload-aware model was given the training-only CTB calibrations required by the application semantics: the sequential observed-variant control-flow contract, calibrated training marginals for routing, an empirical minute-resolved arrival model and non-pre-emptive completion after admission. The freely discovered RMG overlap capacities were then clamped to the physical upper bound of three per block. This cap-three bundle is the frozen effective baseline used for the final validation and the scenarios. Its authoritative KPIs are consolidated in Table 5.10.

Consistent with the corrected temporal semantics of Section 3.2.5, the real event log contains only observed timestamps (`start:timestamp` and `time:timestamp`); activity enablement

is derived internally by ProSiT from A* alignment against the discovered Petri net. Consequently, service-time and case-turnaround comparisons are directly available (both real and simulated logs carry start/complete timestamps), while the model-derived pre-service delay is available only from the simulated log.

Table 5.5 reports the pre-calibration configuration-screening metrics with 95 % Student-t confidence intervals. These values must not be substituted for the final calibrated-baseline values in Table 5.10.

Table 5.5: Pre-calibration held-out screening by ProSiT configuration ($N_{\text{seeds}} = 10$; 95 % Student-t CI).

Metric (min)	no-rules	rules+workload
Case turnaround EMD	13.94 [13.15, 14.73]	21.75 [20.60, 22.90]
Inter-arrival EMD	0.319 [0.317, 0.320]	0.314 [0.313, 0.315]
Case turnaround KS	0.103 [0.099, 0.108]	0.125 [0.123, 0.127]
Sim mean turnaround	53.68 [52.89, 54.47]	61.49 [60.34, 62.64]
Sim P90 turnaround	103.05 [101.26, 104.84]	124.44 [121.70, 127.17]

5.4.1 RQ1: POINT ESTIMATE OF HELD-OUT FIDELITY

The frozen calibrated simulator is evaluated in ten replications of 17,892 cases using seeds 42–51 and the final cutoff $t_{\text{start}} = 2026-04-20\ 18:17$. Its yard-only unweighted service-time EMD is 2.999 [2.888, 3.110] minutes and its real-frequency-weighted yard EMD is 2.340 [2.279, 2.402] minutes. Gate In and Gate Out are excluded. The case-turnaround EMD is 9.306 [9.229, 9.384] minutes and inter-arrival EMD is 0.0724 [0.0674, 0.0774] minutes. Real mean turnaround is 39.742 minutes compared with a simulated 30.436 [30.359, 30.514] minutes; real P90 is 71.0 minutes compared with a simulated 51.800 [51.498, 52.102] minutes. The residual error is therefore a systematic *under-prediction*, not an upper-tail explosion.

All ten runs contain zero gate-only cases, zero within-case overlaps, zero invalid case boundaries, zero decreasing completion sequences and zero cases in which Gate Out precedes the final yard activity. No service event or case turnaround exceeds 24 hours. The final model is thus structurally valid and stable, while its duration bias remains an explicit limitation.

5.4.2 H2: DOES THE DATA-AWARE CONFIGURATION IMPROVE FIDELITY?

The comparison between *no-rules* and *rules+workload* directly tests H2. On every fidelity metric except inter-arrival, the distribution-only configuration produces strictly lower (better) values, and the 95 % confidence intervals are disjoint:

- Case turnaround EMD: *no-rules* 13.94 [13.15, 14.73] vs. *rules+workload* 21.75 [20.60, 22.90]. The CIs are disjoint; *rules+workload* is 56 % worse.
- Inter-arrival EMD: *rules+workload* 0.314 [0.313, 0.315] vs. *no-rules* 0.319 [0.317, 0.320]. Here the tree-based configuration wins, but the absolute difference is 0.005 minutes ($\approx$ 0.3 seconds).

H2 is rejected. The data-aware configuration does not improve fidelity on the CTB event log. The distribution-only simulator achieves a 56 % lower case-turnaround EMD with disjoint confidence intervals. The only gain for the tree-based variant is on the inter-arrival distribution and it is operationally negligible.

5.4.3 H3: DOES THE WORKLOAD-FEATURE CONFIGURATION IMPROVE FIDELITY?

H3 cannot be tested in isolation because we compare *no-rules* (the distribution-only baseline) against *rules+workload* (the full data-aware variant). Since the combined effect of enabling both trees and workload features produces strictly worse fidelity than the no-rules baseline, H3 is subsumed by the H2 rejection. Workload features do not compensate for the fidelity loss introduced by the decision trees.

H3 is rejected. Adding queue-length and workload features to the ProSiT waiting-time and resource-selection models does not improve held-out fidelity on the CTB event log.

5.4.4 CROSS-SEED VARIANCE DOES NOT FAVOUR THE WORKLOAD VARIANT

The seed-to-seed standard deviation of the *rules+workload* screening configuration is not lower than that of *no-rules*. Table 5.6 lists the standard deviation of the headline metrics across the ten seeds.

Table 5.6: Cross-seed standard deviation of the fidelity metrics ($N_{\text{seeds}} = 10$).

Metric	no-rules	rules+workload
Case turnaround EMD (min)	1.10	1.61
Case turnaround sim P90 (min)	2.50	3.82

The *no-rules* configuration produces lower cross-seed variance than *rules+workload* on every listed metric. The *rules+workload* simulator has turnaround-EMD standard deviation 46 % larger than *no-rules* (1.61 vs. 1.10 minutes). Consequently, neither mean fidelity nor sampling variance provides evidence for a workload-feature advantage in this screening experiment. The workload-aware bundle is retained for the final white-box case study because it exposes the context-dependent parameter logic required for scenario intervention, and is subsequently calibrated using training data only.

5.4.5 METHODOLOGICAL NOTE ON TEMPORAL SEMANTICS

The event log used for discovery and validation contains only observed timestamps (`start:timestamp` and `time:timestamp`). Activity enablement is derived internally by ProSiT via the A*-based optimal alignment defined in Section 2.3.1 against the discovered Petri net. This means:

- Service-time EMD compares real vs. simulated complete — start per activity — both quantities are directly available.
- Case-turnaround EMD compares real vs. simulated last_complete — first_start per case — both quantities are directly available.
- Pre-service delay (ProSiT “waiting time”) is available only from the simulated log, where ProSiT emits the model-derived `enabled:timestamp`. A cross-log comparison of waiting time is not performed because the real log does not contain an observed enablement timestamp.

The configuration-screening result is not the final baseline result. In particular, its single-run tail behaviour must not be transferred to the calibrated model, whose ten-seed validation instead shows systematic turnaround under-prediction (Table 5.10).

5.5 PER-ACTIVITY DECOMPOSITION OF FIDELITY

The aggregate fidelity numbers of Section 5.4 hide the per-activity performance of the ProSiT trees. To locate where data-aware conditioning does and does not add value, Table 5.7 reports the coefficient of determination R^2 of the ProSiT-style decision-tree grid used in the sensitivity study `05_data_aware_discovery_evaluation.py` for each activity on the training partition.

Table 5.7: Per-activity duration R^2 of a decision-tree model conditioned on target demand, target utilisation, target rank, visit complexity, receive/delivery counts, hour, weekday and weekend indicator (training log; grid search over depth/leaf, best per activity).

Activity	R^2
LL_delivery	0.263
LL_mixed	0.258
HO ₂ _receive	0.115
HO ₂ _mixed	0.070
RMG_mixed	0.051
HO ₂ _delivery	0.029
LL_receive	0.024
RMG_receive	-0.005
RMG_delivery	-0.004
Gate In	1.000
Gate Out	1.000

Gate activities have $R^2 = 1.000$ because their service duration is constant zero by construction: Gate In and Gate Out have identical start and completion timestamps (Section 3.2.5). For the seven yard activities the R^2 ranges from -0.005 (RMG activities) to 0.263 (LL_delivery). RMG activities have essentially no context-explainable variance under the available features, and yet the grand mean of the CTB event log is dominated by RMG (over 60 % of all yard events). This explains the H2 result of Section 5.4: the ProSiT trees are asked to predict activities whose duration variance is not explained by the available case-level attributes, and the resulting leaf-level distribution fits are noisier than the empirical marginal fit that *no-rules* uses. The overall fidelity metric of Section 5.4 is therefore an aggregate in which the RMG activities (large weight, no context-explainable variance) dominate the LL activities (small weight, meaningful context-explainable variance).

5.6 FREELY DISCOVERED RESOURCE REPRESENTATION AND FINAL DOMAIN CAP

The frozen, freely discovered source bundle contains 26 resource identifiers and the maximum-concurrency values in Table 5.8. These source values are retained as discovery provenance. The final validation and scenario runs load a derived executable copy in which every RMG block is capped at three; all other discovered parameters remain unchanged. The earlier pool-level report produced by `baseline/04_resource_concurrency_discovery.py` is an exploratory diagnostic and is not the authoritative simulator configuration.

Table 5.8: Resource representation in the freely discovered calibrated source bundle before applying the final RMG cap.

Family	Identifiers	Eligible activities	Maximum concurrency
Gate In	Res.GateIn	Gate In	1
Gate Out	Res.GateOut	Gate Out	1
HO ₂	H02	three HO ₂ activities	15
LL	LL	three LL activities	22
RMG	T06–T27 (22 blocks)	three RMG activities	4–5 ^a

^aTen RMG block identifiers have maximum concurrency four and twelve have maximum concurrency five; their aggregate upper bound is therefore 100 simultaneous RMG tasks. This exceeds the nominal physical upper bound of 66 implied by at most three cranes at each of 22 blocks.

All 168 weekly slots of every final resource calendar are available; temporal gate closure is represented separately by the arrival calendar. Each RMG activity can select all 22 block identifiers, and the resource-selection rules supply statistical weights. Closing T22 thus leaves 21 eligible identifiers and removes three units of maximum concurrency from the final cap-three baseline. The freely discovered aggregate bound of 100 motivates the domain constraint reported in Section 5.8: the source object remains frozen for audit, while the executable final baseline clamps every block to three.

These values are learned effective server capacities, not verified numbers of cranes, workers or physically interchangeable storage positions. The 100-versus-66 discrepancy demonstrates that concurrent event intervals assigned to a block cannot be interpreted one-to-one as simultaneously operating cranes. In addition, an RMG block label functions as a selectable resource; it does not create container-location state. Consequently, the T22 reassignment is independent of whether the corresponding container could physically be stored or reached at the replace-

ment block. These are central boundaries for interpreting Scenario A and are carried forward explicitly in Section 5.10.

5.7 DESCRIPTIVE ANALYSIS OF TRANSITION DURATIONS

As noted in Section 3.2.5, an earlier pipeline version used route-specific transition baselines to reconstruct activity enablement. Although this approach was superseded by ProSiT’s internal model-derived enablement, the empirical transition durations remain informative because they characterise the physical-movement component that is embedded within ProSiT’s model-derived pre-service delay.

The analysis of Section 4.8 recomputes the baseline table using percentiles $\{5, 10, 25, 50\}$ and reports the mean shift and the rank correlation. Table 5.9 summarises the result over the 48 transitions with at least twenty observations.

Table 5.9: Empirical transition duration under alternative percentiles (seconds; 48 transitions with ≥ 20 observations).

Percentile	Mean (s)	Median (s)	Mean shift (s)	P90 shift (s)	Corr vs. p5
p5	225.8	223.5	0.0	0.0	1.000
p10	254.6	254.3	28.8	36.0	0.990
p25	318.3	311.0	92.5	110.3	0.948
p50	435.5	426.0	209.8	250.9	0.812

Two properties are relevant for interpreting the model-derived pre-service delay. First, the ranking of transitions by duration is almost invariant to the percentile choice: ρ between p5 and p10 is 0.990, and even between p5 and p50 it is 0.812. Second, the absolute shift grows quickly: moving from the 5th to the 50th percentile shifts the mean transition duration by 3.5 minutes. Because physical truck movement is not modelled as a separate activity, this shift is entirely absorbed into ProSiT’s model-derived pre-service delay.

Figure 5.1 shows the CDF of the per-transition shift across the 48 retained transitions.

Interpretation. The transition-duration analysis confirms that the model-derived pre-service delay reported in Section 5.4 includes a non-trivial physical-movement component (at least 3.5 minutes on average at the median percentile). This does not invalidate the ProSiT waiting-time model — it identifies a domain-specific limitation: the “waiting time” reported in the CTB

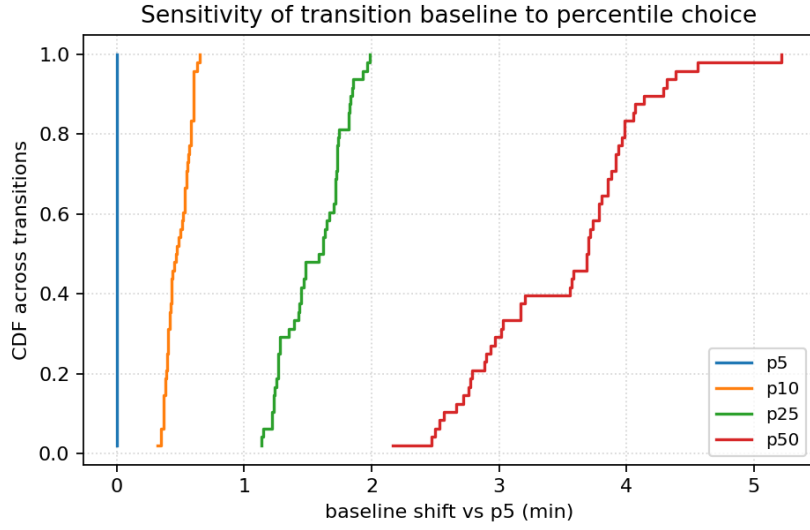


Figure 5.1: CDF of the per-transition duration shift relative to the 5th percentile, over the 48 transitions with at least twenty observations. Illustrates the physical-movement component embedded in the model-derived pre-service delay.

context is broader than pure queueing and should be interpreted as pre-service delay. This limitation is discussed in Section 5.10.

5.8 FINAL CALIBRATED MODEL AND WHAT-IF SCENARIOS

The authoritative scenario baseline is the final sequential, training-only calibrated *rules+workload* parameter bundle with every RMG resource capped at the physical upper bound of three concurrent tasks. The freely discovered values of four or five are retained as diagnostic provenance, not used as the final operational baseline. The hold-out was used only for validation. Ten matched seeds (42–51) were run for the baseline, the T22 resource-pool reallocation and the independent 20 % demand increase, giving 30 complete simulations of 17,892 cases each. Table 5.10 is the single authoritative numerical summary of the final model and both scenarios. For held-out validation rows, the last column is the 95 % Student-t interval of the estimate; for scenario rows it is the 95 % interval of the paired change.

Table 5.10: Consolidated final validation and scenario KPIs ($N_{\text{seeds}} = 10$, seeds 42–51; 17,892 cases per run). Pre-service denotes the model-derived interval from enablement to activity start, not pure queueing time. “Observed” arrival measures use the minute-resolved simulated event log and elapsed calendar time.

Analysis	KPI	Reference	Outcome	Change	95 % CI
<i>Held-out fidelity of the final cap-three baseline</i>					
Validation	Yard service EMD, unweighted (min)	–	2.999	–	[2.888, 3.110]
Validation	Yard service EMD, real-frequency-weighted (min)	–	2.340	–	[2.279, 2.402]
Validation	Case-turnaround EMD (min)	–	9.306	–	[9.229, 9.384]
Validation	Case-turnaround KS	–	0.1554	–	[0.1517, 0.1591]
Validation	Inter-arrival EMD (min)	–	0.0724	–	[0.0674, 0.0774]
Validation	Inter-arrival KS	–	0.0254	–	[0.0215, 0.0292]
Validation	Mean turnaround: real / simulated (min)	39.742	30.436	–9.306	[30.359, 30.514] ^a
Validation	P90 turnaround: real / simulated (min)	71.000	51.800	–19.200	[51.498, 52.102] ^a
Validation	Simultaneous-arrival share: real / simulated (%)	56.660	54.221	–2.439 pp	[54.035, 54.407] ^a
Validation	Yard-activity-rate L1 error	–	0.1866	–	[0.1838, 0.1893]
<i>Scenario A: T22 resource-pool reallocation (scenario minus matched baseline)</i>					
T22 pool	Mean turnaround (min)	30.443	30.456	+0.012	[–0.102, +0.127]
T22 pool	P90 turnaround (min)	51.700	51.900	+0.200	[–0.364, +0.764]
T22 pool	Mean RMG service (min)	11.069	11.059	–0.010	[–0.064, +0.045]
T22 pool	Mean RMG pre-service (min)	6.475	6.479	+0.003	[–0.034, +0.040]
T22 pool	RMG receive service (min)	11.362	11.319	–0.043	[–0.110, +0.024]
T22 pool	RMG receive pre-service (min)	6.480	6.496	+0.016	[–0.039, +0.071]
T22 pool	RMG delivery service (min)	8.794	8.829	+0.035	[–0.078, +0.147]
T22 pool	RMG delivery pre-service (min)	6.487	6.451	–0.036	[–0.087, +0.015]
T22 pool	Mean T22 assignments per run	604.8	0.0	–604.8	exact
<i>Scenario B: 20 % higher working-time arrival intensity (scenario minus matched baseline)</i>					
Demand +20 %	Model mean working inter-arrival (min)	0.6830	0.5692	–0.1138	exact
Demand +20 %	Model arrivals per working hour	87.842	105.411	+20.000 %	exact
Demand +20 %	Observed mean inter-arrival (min)	0.8282	0.7145	–0.1137	[–0.1145, –0.1128]
Demand +20 %	Observed arrivals per elapsed hour	72.452	83.979	+11.527 (+15.910 %)	[+11.478, +11.577]
Demand +20 %	Observed simultaneous-arrival share (%)	54.221	60.015	+5.795 pp	[+5.736, +5.853] pp
Demand +20 %	Mean turnaround (min)	30.443	30.411	–0.033	[–0.128, +0.063]
Demand +20 %	Mean RMG service (min)	11.069	11.063	–0.006	[–0.067, +0.056]
Demand +20 %	Mean RMG pre-service (min)	6.475	6.429	–0.046	[–0.094, +0.001]
Demand +20 %	RMG receive service / pre-service (min)	11.362 / 6.480	11.332 / 6.461	–0.030 / –0.020	[–0.101, +0.041] / [–0.074, +0.035]
Demand +20 %	RMG delivery service / pre-service (min)	8.794 / 6.487	8.796 / 6.397	+0.002 / –0.090	[–0.083, +0.087] / [–0.157, –0.022]
Structure	Aggregate RMG maximum concurrency	100 discovered	66 final	–34	exact domain cap
Contract	Invalid cases / duration tails	40 final logs	0	0	all checks passed

^aFor these rows the interval refers to the simulated outcome; the signed change is outcome minus the fixed real-holdout statistic. Scenario intervals refer to paired seed-wise changes. Gate In and Gate Out are excluded from both yard-service EMD aggregates.

Table 5.11: Held-out service-time EMD by yard activity for the final cap-three baseline ($N_{\text{seeds}} = 10$). Gate In and Gate Out are excluded.

Yard activity	Mean EMD (min)	95 % CI
H02_delivery	2.119	[2.013, 2.225]
H02_mixed	4.255	[3.853, 4.656]
H02_receive	2.024	[1.768, 2.280]
LL_delivery	1.353	[1.286, 1.420]
LL_mixed	3.857	[3.019, 4.695]
LL_receive	4.203	[4.000, 4.406]
RMG_delivery	2.519	[2.426, 2.612]
RMG_mixed	4.200	[3.971, 4.429]
RMG_receive	2.463	[2.374, 2.551]

5.8.1 HELD-OUT BASELINE INTERPRETATION

The final baseline reproduces the nine yard-service distributions with an unweighted EMD of 2.999 minutes. Weighting the same per-activity distances by real held-out frequency reduces the value to 2.340 minutes, because the rarer mixed and LL-receive activities have the largest distances (Table 5.11). Arrivals have an EMD of 0.072 minutes. Case-turnaround EMD is 9.306 minutes. This discrepancy is systematic rather than Monte-Carlo noise: mean turnaround is under-predicted by 9.306 minutes and P90 by 19.200 minutes. The narrow confidence intervals quantify simulation sampling uncertainty for this one frozen model; they do not quantify model-discovery uncertainty. Crucially, all ten validation replications satisfy the sequential case contract and contain no service event or turnaround above 24 hours.

5.8.2 SCENARIO A: T22 RESOURCE-POOL REALLOCATION

T22 handles on average 604.8 assignments per final-baseline replication, or 4.41 % of RMG events. The intervention reduces this count to zero. With every remaining block capped at three, neither the mean-turnaround change of +0.012 $[-0.102, +0.127]$ minutes nor the overall RMG service/pre-service changes exclude zero.

Separating operation types does not reveal a hidden material effect. Receive service changes by -0.043 $[-0.110, +0.024]$ minutes and receive pre-service by $+0.016$ $[-0.039, +0.071]$ minutes. Delivery service changes by $+0.035$ $[-0.078, +0.147]$ minutes and delivery pre-service by -0.036 $[-0.087, +0.015]$ minutes. All four intervals cross zero. The model therefore supports

the conclusion that removing T22 from its abstract RMG resource pool has no statistically resolved differential effect on receive or delivery operations in this experiment.

The effect must not be interpreted as a physically complete yard-block closure study. All three RMG activities retain 21 eligible blocks, each with maximum concurrency three. More importantly, the resource selector is not a container-location state model: removal of T22 causes statistical reassignment to another eligible resource even when the historical target area was T22. Container relocation, block compatibility and the additional physical travel caused by relocation are not represented. The result therefore measures robustness of the *abstract discovered resource pool*, not the full operational consequence of closing the physical T22 storage block.

5.8.3 SCENARIO B: 20 % HIGHER DEMAND

The white-box parameter intervention is exact in working time: the empirical mean inter-arrival changes from 0.6830 to 0.5692 minutes and the implied intensity from 87.842 to 105.411 arrivals per working hour. The observed calendar-time rate rises by 15.910 %, rather than 20 %, because closed gate periods occupy fixed elapsed time and the event log is observed after flooring timestamps to minutes. The same flooring maps additional sub-minute positive intervals to simultaneous observations, raising the observed zero-inter-arrival share while the zero mass of the input distribution itself remains unchanged.

Despite the higher demand, neither mean turnaround nor overall RMG service or pre-service delay changes significantly in the final cap-three model: mean turnaround changes by -0.033 $[-0.128, +0.063]$ minutes. Receive service and pre-service also remain unresolved. Delivery pre-service decreases by -0.090 $[-0.157, -0.022]$ minutes (approximately 5.4 seconds), while delivery service is unchanged. This small counter-intuitive effect is compatible with a changed routing/resource-selection mix, but it is not evidence of causal delivery prioritisation: ship-side crane allocation, physical container locations and explicit queue states are absent. The result instead shows that a cap of three alone does not create the expected congestion response in the current abstraction.

This is a legitimate scenario result, but also a warning: because spatial container constraints and explicit equipment movements are absent, it cannot by itself establish that the physical terminal would absorb 20 % more demand without congestion.

H4 is partially supported. Both direct interventions are executed as specified: T22 assignments fall to zero and the demand parameter increases working-time arrival intensity by exactly 20 %.

The T22 resource-pool reallocation produces no statistically resolved overall or receive-versus-delivery effect. The demand experiment yields a small delivery pre-service reduction but no statistically detectable mean-turnaround effect. H4 therefore does not support a strong operational decision recommendation.

5.9 SUMMARY OF EMPIRICAL FINDINGS

Table 5.12 consolidates the empirical results against the hypotheses of Section 4.1.2.

Table 5.12: Summary of empirical findings.

Hypothesis	Evidence	Outcome
H1 (ar-rival time-dependence)	Time-aware tree reduces MAE by 68.8 % and RMSE by 52.8 % versus pooled baseline; $R^2 = 0.777$ (Table 5.3).	Accepted.
H2 (data-aware fidelity gain)	<i>rules+workload</i> does not lower turnaround EMD versus <i>no-rules</i> on the held-out log; in fact, the distribution-only simulator achieves 56 % lower turnaround EMD with disjoint 95 % CIs (Table 5.5).	Rejected.
H3 (workload-feature gain)	H3 is subsumed by the H2 rejection: the combined effect of trees and workload features produces strictly worse fidelity than the distribution-only baseline.	Rejected.
H3-secondary (sampling variance)	<i>rules+workload</i> has larger, not smaller, cross-seed standard deviation of turnaround EMD than <i>no-rules</i> (1.61 vs. 1.10 minutes; Table 5.6).	Does not offset H2 rejection.
H4 (scenario response)	T22 assignments fall to zero, but none of its overall or receive-versus-delivery timing effects exclude zero. The demand parameter increases working-time intensity by exactly 20 %; only the small delivery pre-service reduction excludes zero, and neither intervention has a significant mean-turnaround effect (Table 5.10).	Partially supported; no strong decision claim.

Section 5.2 (RQ₂) establishes that the Petri net generalises to the held-out period. Section 5.4 (RQ₁) reports that the final calibrated ProSiT simulator under-predicts case-mean turnaround by 9.306 minutes (23.42 %) and P90 by 19.200 minutes (27.04 %). Section 5.5 decomposes the remaining error across activities, whose context-explainable variance is essentially zero under the available features. Section 5.7 provides descriptive evidence that the model-derived pre-service delay includes a non-trivial physical-movement component (3.5 minutes median shift between percentiles), confirming the domain-specific limitation discussed in Section 5.10.

5.10 METHODOLOGICAL LIMITATIONS

5.10.1 MODEL-DERIVED WAITING TIME INCLUDES PHYSICAL MOVEMENT

ProSiT follows process-model semantics in which an activity becomes enabled when the control-flow state permits its execution. In the CTB process, however, consecutive operational activities occur at spatially separated terminal locations. Physical truck movement between these locations is not represented as a separate activity in the discovered process model. Consequently, the model-derived interval between enablement and activity start includes both physical transition time and operational waiting:

$$D_{\text{pre-service}} \approx T_{\text{physical}} + T_{\text{traffic}} + T_{\text{queue}} + T_{\text{other}}. \quad (5.1)$$

These components are not individually identifiable from the available data. Individual truck travel times are not directly observed; the scheduling-system transition durations combine movement with congestion and other operational effects. Separating the components would require position or trajectory data not available in the CTB systems. The thesis deliberately retains the standard ProSiT semantics rather than adding an unobserved transition-time model.

Waiting-time results should therefore be interpreted as *pre-service delay* rather than as a direct measurement of queueing or equipment-induced waiting.

5.10.2 RESOURCE ASSIGNMENT IS NOT SPATIAL CONTAINER RELOCATION

The RMG resource selected by ProSiT is an abstract capacity assignment. Its freely discovered aggregate overlap capacity is 100, whereas the stated physical maximum of three cranes per

block implies at most 66 across 22 blocks. This mismatch is evidence that timestamp overlap also absorbs event-log and process abstractions; it is not a physical crane count. The final executable baseline therefore applies the cap of three. Even under that bound, the 20 % demand scenario produces no significant mean-turnaround increase. Thus excessive overlap capacity is not the only missing mechanism; spatial state and waterside/landside equipment allocation remain unmodelled.

Although target-area attributes can enter a discovered decision tree, they are predictors rather than a hard container-inventory constraint. The simulator does not maintain a state that binds each container to its actual storage block, nor does it create a relocation activity when the selected block differs from that historical location. Consequently, closing T22 redistributes its statistical resource demand across the remaining eligible blocks without modelling relocation feasibility, yard-crane repositioning or the resulting change in physical truck travel. The T22 experiment is valid as a white-box test of the discovered resource-pool abstraction, but not as a stand-alone forecast of the physical closure of yard block T22.

5.10.3 SYSTEMATIC TURNAROUND UNDER-PREDICTION

The final calibrated model under-predicts mean turnaround by 9.306 minutes and P90 by 19.200 minutes. The narrow cross-seed intervals show that this is systematic model bias rather than simulation noise. The available event log cannot identify how much of the missing time is physical movement, traffic, queueing or an unrecorded operational state. This limits absolute KPI forecasting even though the generated case order and duration tails satisfy the formal validation contract.

5.10.4 SCENARIO CONFIDENCE INTERVALS EXCLUDE STRUCTURAL UNCERTAINTY

The matched-seed intervals in Table 5.10 quantify random simulation variation conditional on one frozen discovered model. They do not cover uncertainty from repeating discovery on another training window or adding the missing spatial container state. The cap-three domain constraint enforces a physical ceiling, but does not represent resource-location feasibility or waterside/landside equipment movement. Increasing the number of seeds would narrow sampling intervals but would not resolve those remaining structural sources of uncertainty.

5.II INTERPRETIVE CONCLUSION OF THE RESULTS CHAPTER

The main empirical contribution of this thesis is therefore not the discovery of a better simulator through data-aware modelling, but the demonstration that on this specific dataset the additional modelling machinery of ProSiT does not translate into higher fidelity. The observation, together with the multi-seed protocol that reveals it and the absence of workload-feature variance reduction, is the operational finding that Chapter 6 builds upon.

6

Conclusion (3)

This thesis addressed one of the most important open challenges in ...

The theoretical contribution of this research was ...

Finally, we demonstrated how our new framework...

Ultimately, this thesis provides both a...

6.1 FUTURE WORK

While the proposed framework establishes a solid foundation for... To advance this framework, future efforts should be directed across the following key areas:

- Different TErminAl Setting): A fundamental structural difference between SG

6.2 PERSONAL TAKEAWAYS

Beyond the theoretical conclusions of this thesis, I would like to close by reflecting on the personal and professional insights I take away from this research project:

1. Independent Project Management: I learned how to self-organize, manage timelines effectively, and work independently to drive a large-scale project to completion.
2. Embracing Setbacks as Progress: I understood firsthand that failure, unexpected results, and rigorous debugging are not roadblocks, but rather inherent and necessary steps in the scientific research process.
3. The Value of First Principles Thinking: I learned that when facing complex challenges, the most effective strategy is often to step back, re-evaluate the fundamental mathematics and logic, and approach the problem from a renewed perspective.
4. Prioritizing Reproducibility Over Complexity: I realized that a tested, reproducible, and well-documented framework holds far more scientific value than a system that is overly complex but unmanageable. Furthermore, working in small, modular, and heavily tested scripts proved absolutely necessary when scaling up to a large codebase.
5. Navigating Collaborative Research Dynamics: I experienced the immense value of working within an active research group, where navigating open questions and integrating diverse feedback significantly enriched the final work.

References

- [1] W. Van der Aalst, *Process Mining: Data Science in Action*, 01 2016.
- [2] W. Y. Yun and Y. S. Choi, "A simulation model for container-terminal operation analysis using an object-oriented approach," *International Journal of Production Economics*, vol. 59, no. 1, pp. 221–230, 1999. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925527398002138>
- [3] D. Steenken, S. Voss, and R. Stahlbock, "Container terminal operation and operations research - a classification and literature review," *OR Spectrum*, vol. 26, pp. 3–49, 01 2004.
- [4] B. Weerasinghe, N. Perera, and X. Bai, "Optimizing container terminal operations: a systematic review of operations research applications," *Maritime Economics and Logistics*, vol. 26, pp. 1–35, 02 2023.
- [5] D. Bett, I. Ali, M. Gheith, and A. Eltawil, "Simulation-based optimization of truck appointment systems in container terminals: A dual transactions approach with improved congestion factor representation," *Logistics*, vol. 8, p. 80, 08 2024.
- [6] A. Rozinat, R. Mans, M. Song, and W. Aalst, "Discovering simulation models," *Information Systems*, vol. 34, pp. 305–327, 05 2009.
- [7] V. Leno, M. Dumas, F. Maggi, M. La Rosa, and A. Polyvyanyy, "Automated discovery of declarative process models with correlated data conditions," *Information Systems*, vol. 89, p. 101482, 12 2019.
- [8] F. Mannhardt, S. J. J. Leemans, C. T. Schwanen, and M. de Leoni, "Modelling data-aware stochastic processes - discovery and conformance checking," in *Application and Theory of Petri Nets and Concurrency: 44th International Conference, PETRI NETS 2023, Lisbon, Portugal, June 25–30, 2023, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2023, p. 77–98. [Online]. Available: https://doi.org/10.1007/978-3-031-33620-1_5

- [9] M. {de Leoni}, F. Vinci, S. Leemans, and F. Mannhardt, “Investigating the influence of data-aware process states on activity probabilities in simulation models: Does accuracy improve?” in *Business Process Management*, ser. Lecture Notes in Computer Science (LNCS), C. {Di Francescomarino}, A. Burattin, C. Janiesch, and S. Sadiq, Eds. Germany: Springer, Sep. 2023, pp. 129–145, 21st International Conference on Business Process Management, BPM 2023, BPM 2023 ; Conference date: 11-09-2023 Through 15-09-2023.
- [10] O. López-Pintado, S. Murashko, and M. Dumas, “Discovery and simulation of data-aware business processes,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.13666>
- [11] F. Meneghello, C. Di Francescomarino, C. Ghidini, and M. Ronzani, “Runtime integration of machine learning and simulation for business processes: Time and decision mining predictions,” *Information Systems*, vol. 128, p. 102472, 10 2024.
- [12] F. Vinci, G. Park, W. Aalst, and M. de Leoni, *Reliable and Configurable Process Simulations via Probabilistic White-Box Models*, 01 2026, pp. 337–352.
- [13] F. Vinci, D. Aloini, E. Benevento, A. Stefanini, F. Zen, and M. de Leoni, “Improving organizational processes in healthcare through simulation-driven resource allocation: Methodology and real-world case study,” *Artificial Intelligence in Medicine*, vol. 176, p. 103391, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0933365726000436>
- [14] M. Camargo, M. Dumas, and O. González-Rojas, “Automated discovery of business process simulation models from event logs,” *Decision Support Systems*, vol. 134, p. 113284, 03 2020.
- [15] S. Hartmann, *Generating scenarios for simulation and optimization of container terminal logistics*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 101–122. [Online]. Available: https://doi.org/10.1007/3-540-26686-0_4
- [16] M. Gracia, J. Mar-Ortiz, C. Caballini, and S. Sacone, “A combined data mining – optimization approach to manage trucks operations in container terminals with the use of a tas: Application to an italian and a mexican port,” *Transportation Research Part E Logistics and Transportation Review*, vol. 142, 08 2020.

- [17] M. Kastner, N. Nellen, A. Schwientek, and C. Jahn, "Integrated simulation-based optimization of operational decisions at container terminals," *Algorithms*, vol. 14, 01 2021.
- [18] A. Carboni, F. Defflorio, C. Caballini, and S. Cangelosi, "Advances in terminal management: Simulation of vehicle traffic in container terminals," *Maritime Economics & Logistics*, vol. 27, no. 3, pp. 500–524, 2025. [Online]. Available: <https://doi.org/10.1057/s41278-024-00300-5>
- [19] D. Muravev, A. Rakhmangulov, H. Hu, and H. Zhou, "The introduction to system dynamics approach to operational efficiency and sustainability of dry port's main parameters," *Sustainability*, vol. 11, p. 2413, 04 2019.
- [20] D. Muravev, H. Hu, A. Rakhmangulov, and P. Mishkurov, "Multi-agent optimization of the intermodal terminal main parameters by using anylogic simulation platform: Case study on the ningbo-zhoushan port," *International Journal of Information Management*, vol. 57, p. 102133, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S026840121931789X>
- [21] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Discovering block-structured process models from event logs containing infrequent behaviour," in *Business Process Management Workshops*, ser. Lecture Notes in Business Information Processing, vol. 171. Springer, 2014, pp. 66–78.
- [22] R. de la Briandais, "File searching using variable length keys," in *Proceedings of the Western Joint Computer Conference*. ACM, 1959, pp. 295–298.
- [23] S. A. Fahrenkrog-Petersen, H. van der Aa, and M. Weidlich, "Pretsa: Event log sanitization for privacy-aware process discovery," in *2019 International Conference on Process Mining (ICPM)*. IEEE, 2019, pp. 1–8.
- [24] W. Aalst, A. Adriansyah, and B. Dongen, "Replaying history on process models for conformance checking and performance analysis," *WIREs Data Mining and Knowledge Discovery*, vol. 2, pp. 182–192, 03 2012.
- [25] A. Adriansyah, B. F. van Dongen, and W. M. P. van der Aalst, "Conformance checking using cost-based fitness analysis," in *2011 IEEE 15th International Enterprise Distributed Object Computing Conference*. IEEE, 2011, pp. 55–64.

- [26] W. Aalst, “Business process simulation survival guide,” pp. 337–370, 04 2015.
- [27] A. Bruzzone, F. Longo, L. Nicoletti, E. Bottani, and R. Montanari, “Simulation, analysis and optimization of container terminals processes,” *International Journal of Modeling, Simulation, and Scientific Computing*, vol. 03, p. 1240006, 11 2012.
- [28] D. Chapela-Campa, M. Dumas, O. Pintado, and I. Suvorau, “Simod: Automated discovery of business process simulation models,” 01 2024.
- [29] O. Pintado and M. Dumas, *Business Process Simulation with Differentiated Resources: Does it Make a Difference?*, 09 2022, pp. 361–378.
- [30] M. Camargo, M. Dumas, and O. González-Rojas, “Discovering generative models from event logs: data-driven simulation vs deep learning,” *PeerJ Computer Science*, vol. 7, p. e577, 07 2021.
- [31] D. Chapela-Campa, I. Bencheikroun, O. Baron, M. Dumas, D. Krass, and A. Senderovich, “A framework for measuring the quality of business process simulation models,” *Information Systems*, vol. 127, p. 102447, 01 2025.
- [32] S. Leemans, A. Syring, and W. Aalst, *Earth Movers’ Stochastic Conformance Checking*, 07 2019, pp. 127–143.
- [33] Hamburger Hafen und Logistik AG, “Geschäftsbericht 2025,” Hamburger Hafen und Logistik AG, Hamburg, Tech. Rep., 2026. [Online]. Available: https://bericht.hhla.de/geschaeftsbericht-2025/_assets/downloads/HHLA_2025_Geschaeftsbericht.pdf

Acknowledgments

Thank you to Professor Esteban Zimanyi for pouring his heart into this Masters Program and eventually a little bit into this Thesis too, even though you didnt have to. Thank you for being the soul of this adventure I was able to go through with this Program.

Thank you to Massimiliano for insisting on Process Mining and opening this world to me and for teaching me how to believe in myself and the process.

Thank you to my fellow BDMAers, for inspiring me and being the gentle souls that you are. And a little special Thank you to Charolette, for keeping my faith up and and by my side, to Josu for the endless hard work and big inspiration that you are and to Alfio.